

02장 tkinter 기본 (tkinter Basic)

tkinter 기본 개요

tkinter는 파이썬에 기본으로 포함되어 있어 별도의 복잡한 설치 과정 없이 GUI 프로그래밍을 시작할 수 있다는 장점이 있습니다.

처음에는 간단한 창과 몇 가지 화면 요소로 출발하지만, 점차 입력, 선택, 배치, 반응 처리와 같은 GUI 프로그램의 핵심 개념을 이해하게 됩니다. 이러한 기본 구조를 익혀 두면 이후 더 복잡한 데스크톱 애플리케이션을 만들 때도 안정적인 기반이 됩니다.

GUI 프로그램에서 중요한 점은 각각의 화면 요소를 따로 아는 것만이 아니라, 이들이 서로 어떻게 연결되어 하나의 프로그램처럼 동작하는지 이해하는 것입니다. 화면 구성 요소를 적절히 배치하고, 사용자의 동작에 따라 필요한 처리를 실행하며, 상황에 맞는 정보를 보여주는 과정을 통해 tkinter 프로그램의 전체적인 동작 방식을 자연스럽게 익힐 수 있습니다.

이번 장의 예제들은 작은 단위로 구성되어 있지만, 각각은 실제 GUI 프로그램을 만들 때 자주 사용되는 기본 패턴을 담고 있습니다. 예제를 따라 하며 옵션을 바꾸고 동작을 수정해 보면, 단순히 코드를 입력하는 수준을 넘어 “왜 이렇게 구성되는가”를 이해하는 데 도움이 됩니다.

이 장을 마치면 tkinter를 이용해 기본적인 GUI 화면을 구성하고, 사용자 입력과 동작에 반응하는 프로그램을 만들 수 있는 기초 역량을 갖추게 됩니다.

목차

- [02-01 GUI 창 띄우기](#)
- [02-02 라벨과 버튼](#)
- [02-03 엔트리와 텍스트](#)
- [02-04 체크, 라디오, 리스트](#)
- [02-05 스피너박스, 스케일](#)
- [02-06 레이아웃](#)
- [02-07 이벤트와 콜백](#)
- [02-08 메뉴](#)
- [02-09 대화상자](#)
- [02-10 캔버스](#)
- [02-11 ttk 위젯과 스타일링](#)

- [02-12 ttk 고유 위젯](#)

GUI 창 띄우기

tkinter로 가장 기본적인 GUI 창을 띄우는 방법을 알아보겠습니다. tkinter 프로그램의 출발점은 창을 만들고, 창을 계속 화면에 유지하는 것입니다.

콘솔 프로그램은 위에서 아래로 실행되고 끝나지만, GUI 프로그램은 사용자가 창을 닫을 때까지 계속 실행됩니다. 이때 창이 사용자 입력을 기다리고, 버튼 클릭이나 창 닫기 같은 이벤트를 처리할 수 있도록 해주는 것이 바로 `mainloop()`입니다.

- [tkinter 프로그램의 기본 구조](#)
 - [가장 단순한 GUI 창 만들기](#)
 - [코드 한 줄씩 살펴보기](#)
 - [mainloop\(\)의 역할](#)
- [창 기본 설정](#)
 - [창 제목 설정](#)
 - [창 크기와 위치 설정](#)
 - [창 배경색 설정](#)
 - [크기 변경 여부](#)
 - [최소/최대 크기](#)

tkinter 프로그램의 기본 구조

가장 단순한 GUI 창 만들기

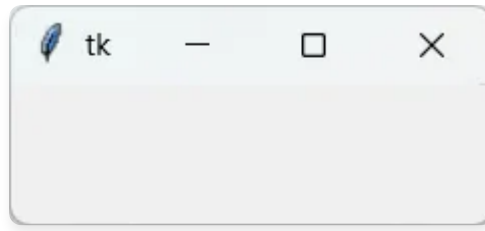
- 먼저 아무 기능도 없는 빈 창을 띄워보겠습니다.

```
import tkinter as tk

root = tk.Tk()

root.mainloop()
```

이 코드는 매우 짧지만, tkinter 프로그램의 핵심 구조를 모두 담고 있습니다.



- tkinter 프로그램은 기본적으로 다음 세 단계로 이루어집니다.

```
tkinter 모듈 가져오기
↓
기본 창 생성하기
↓
mainloop()로 창 유지하기
```

코드 한 줄씩 살펴보기

- tkinter 모듈 가져오기

```
import tkinter as tk
```

- tkinter는 파이썬에서 GUI 프로그램을 만들 때 사용하는 표준 모듈입니다.
- GUI 창, 버튼, 라벨, 입력창 같은 요소들이 모두 이 모듈 안에 들어 있습니다.
- 여기서는 tkinter를 매번 길게 쓰지 않기 위해 tk라는 짧은 이름으로 가져왔습니다.

- 기본 창 생성하기

```
root = tk.Tk()
```

- `tk.Tk()` 는 GUI 프로그램의 기본 창을 만듭니다.
- 이 창을 보통 루트 창(root window) 또는 메인 창이라고 부릅니다.
- root라는 이름은 반드시 써야 하는 것은 아니지만, tkinter 예제에서 가장 흔히 사용하는 관례입니다.

- GUI 프로그램 실행 유지하기

```
root.mainloop()
```

- `mainloop()` 는 tkinter 프로그램을 계속 실행 상태로 유지합니다.
- 이 코드가 없으면 창이 생성되자마자 프로그램이 종료되어 창을 제대로 볼 수 없습니다.
- 즉, tkinter에서 창을 만들었다면 마지막에는 거의 항상 `mainloop()` 를 실행해야 합니다.

알아두기

`mainloop()` 호출 이후에 작성한 코드는 창이 닫혀야 비로소 실행됩니다. 따라서 창 설정과 위젯 배치 코드는 모두 `mainloop()` 위쪽에 작성해야 합니다.

mainloop()의 역할

- 콘솔 프로그램은 보통 위에서 아래로 실행된 뒤 종료됩니다. 하지만 GUI 프로그램은 조금 다르게 동작합니다.
- GUI 프로그램은 사용자가 어떤 행동을 할 때까지 계속 기다립니다.
- 예를 들어 사용자는 다음과 같은 동작을 할 수 있습니다.

창 닫기
버튼 클릭
키보드 입력
마우스 이동
창 크기 변경

이러한 사용자의 동작을 이벤트(event)라고 합니다.

- `mainloop()` 는 이런 이벤트를 계속 감시하고 처리합니다.
- tkinter 프로그램은 다음과 같은 흐름으로 동작합니다.

```
창 생성
↓
위젯 배치
↓
mainloop() 실행
↓
사용자 이벤트 대기
↓
이벤트 발생 시 처리
↓
창을 닫을 때까지 반복
```

- 처음에는 다음 한 문장으로 이해하면 충분합니다.

`mainloop()` 는 GUI 프로그램을 살아 있게 만드는 실행 루프입니다.

창 기본 설정

창 제목 설정

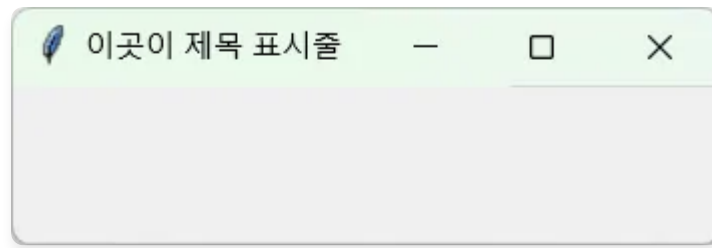
- 기본 창을 만들면 제목 표시줄에 보통 tk라는 이름이 표시됩니다.
- 사용자가 보기 좋도록 창 제목을 바꿔보겠습니다.

```
import tkinter as tk

root = tk.Tk()
root.title("이곳이 제목 표시줄")

root.mainloop()
```

`title()` 메서드는 창의 제목 표시줄에 나타나는 문자열을 설정합니다.



창 크기와 위치 설정

- 창 크기는 `geometry()` 메서드로 설정합니다.

```
import tkinter as tk

root = tk.Tk()
root.geometry("200x50")

root.mainloop()
```

- 여기서 "200x50"은 너비를 200픽셀, 높이를 50픽셀로 설정합니다.

알아두기

`geometry()` 에서 크기를 지정할 때 숫자 사이에는 영어 소문자 x를 사용합니다. 곱하기 기호나 대문자 X를 사용하면 오류가 발생합니다.

- `geometry()` 는 창 크기뿐 아니라 화면에서의 위치도 지정할 수 있습니다.

```
import tkinter as tk

root = tk.Tk()
root.title("창 위치 설정")
root.geometry("400x300+100+200")

root.mainloop()
```


- 형식은 다음과 같습니다.

"가로 x 세로 +x좌표 +y좌표"

위 코드는 창의 왼쪽 위 모서리를 화면의 (100, 200) 위치에 놓는다는 의미입니다.

창 배경색 설정

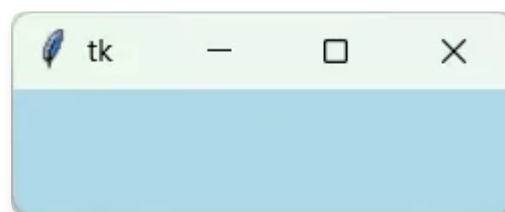
- 창의 배경색은 `configure()` 메서드로 설정합니다.

```
import tkinter as tk

root = tk.Tk()
root.geometry("200x50")
root.configure(bg="lightblue")

root.mainloop()
```

- `bg` 는 background의 줄임말입니다.
- 색상은 16진수 색상 코드도 사용할 수 있습니다.
- `configure()` 대신 짧은 별칭인 `config()` 를 사용해도 동일하게 동작합니다.



크기 변경 여부

- 기본적으로 tkinter 창은 사용자가 마우스로 크기를 변경할 수 있습니다.
- 창 크기 변경을 막고 싶다면 `resizable()` 을 사용합니다.

```
import tkinter as tk

root = tk.Tk()
root.geometry("400x300")
root.resizable(False, False)

root.mainloop()
```

- `resizable()` 은 두 개의 인자를 받습니다. `resizable(가로 변경 여부, 세로 변경 여부)`
- 두 인자 모두 `True` / `False` 로 지정하며, `False` 로 두면 해당 방향의 크기 변경이 잠깁니다.

최소/최대 크기

- 창 크기 변경 자체는 허용하되, 너무 작아지거나 너무 커지는 것을 막고 싶을 수 있습니다.
- 이때는 `minsize()` 와 `maxsize()` 를 사용합니다.

```
import tkinter as tk

root = tk.Tk()
root.geometry("400x300")
root.minsize(300, 200)
root.maxsize(800, 600)

root.mainloop()
```

- 사용자가 창을 줄이더라도 가로 300픽셀, 세로 200픽셀 보다 작게 만들 수 없습니다.
- 사용자가 창을 키우더라도 가로 800픽셀, 세로 600픽셀 보다 크게 만들 수 없습니다.

라벨과 버튼

tkinter에서 기본적으로 사용하는 두 가지 위젯인 **라벨(Label)** 과 **버튼(Button)** 을 살펴보겠습니다.

라벨은 화면에 글자를 표시할 때 사용하고, 버튼은 사용자가 클릭해서 어떤 동작을 실행할 때 사용합니다.

- [Label](#)
 - [Label 사용한 텍스트 출력](#)
 - [코드 한 줄씩 살펴보기](#)
 - [주요 옵션](#)
 - [Label 옵션 적용](#)
- [Button](#)
 - [Button 클릭으로 라벨 변경하기](#)
 - [함수 정의와 호출](#)
 - [command 사용 시 주의할 점](#)
 - [주요 옵션](#)
 - [Button 옵션과 상태 적용](#)

Label

- Label은 창에 텍스트를 표시하는 위젯으로 안내 문구, 상태 메시지, 결과 등을 보여줄 때 사용합니다.

Label 사용한 텍스트 출력

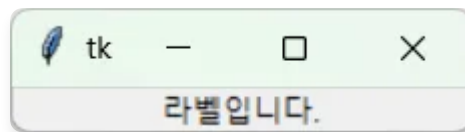
- 윈도우 창에 Label 사용해서 텍스트를 출력하는 코드입니다.

```
import tkinter as tk

root = tk.Tk()

label = tk.Label(root, text="라벨입니다.")
label.pack()

root.mainloop()
```



코드 한 줄씩 살펴보기

- Label이 생성되고 배치되는 핵심 코드는 다음 두 줄입니다.

```
label = tk.Label(root, text="라벨입니다.")
label.pack()
```

- `tk.Label()` 은 라벨을 생성합니다.
- 첫 번째 인자인 `root` 는 라벨이 들어갈 부모 창입니다.
- `text` 옵션에는 화면에 표시할 글자를 넣습니다.
- `pack()` 은 라벨을 창에 배치하는 역할을 합니다.
- 위젯을 생성해도 `pack()` 을 호출하지 않으면 화면에 보이지 않습니다.

주요 옵션

옵션	허용 값	설명
<code>text</code>	문자열	라벨에 표시할 문자열
<code>font</code>	<code>("맑은 고딕", 14, "bold")</code> 형식의 튜플 또는 문자열	글꼴과 글자 크기
<code>fg</code>	색상 이름(<code>"red"</code>) 또는 16진수(<code>"#ff0000"</code>)	글자색
<code>bg</code>	색상 이름(<code>"white"</code>) 또는 16진수(<code>"#ffffff"</code>)	배경색
<code>padx</code>	0 이상의 정수 (픽셀 단위)	좌우 안쪽 여백
<code>pady</code>	0 이상의 정수 (픽셀 단위)	위아래 안쪽 여백

- 예를 들어 다음 코드는 글자색을 흰색으로, 배경색을 남색으로 지정합니다.

```
fg="white",
bg="navy"
```

■ 알아두기

옵션(option)이란?

tkinter의 각 위젯은 다양한 옵션(option)을 통해 모양과 동작을 설정할 수 있습니다. 가령 버튼 위젯에서 버튼에 표시할 문자열을 지정하는 `text`, 색상을 지정하는 `bg` 등이 대표적인 예입니다.

이러한 항목들은 "속성"이라고 표현되기도 하지만, tkinter에서 공식적으로 사용하는 표현은 "옵션"입니다. 따라서 이 책에서는 위젯의 설정 항목을 "옵션"이라는 용어로 설명합니다.

위젯을 생성할 때 옵션을 지정할 수 있고, 생성 후에는 `config()` 또는 `configure()` 메서드로 옵션 값을 변경할 수 있습니다.

Label 옵션 적용

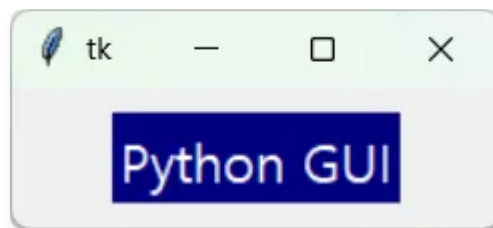
- 라벨 위젯은 글자 크기, 글자색, 배경색 등을 지정할 수 있습니다.

```
import tkinter as tk

root = tk.Tk()

label = tk.Label(
    root,
    text="Python GUI",
    font=("맑은 고딕", 16),
    fg="white",
    bg="navy"
)
label.pack(padx=40, pady=10)

root.mainloop()
```



Button

- `Button` 은 사용자가 클릭할 수 있는 버튼을 만드는 위젯입니다.
- 버튼은 단독으로 존재하기보다, 클릭했을 때 실행할 함수와 함께 사용되는 경우가 많습니다.

Button 클릭으로 라벨 변경하기

- 버튼을 클릭하면 콘솔에 텍스트가 출력되는 코드입니다.

```
import tkinter as tk

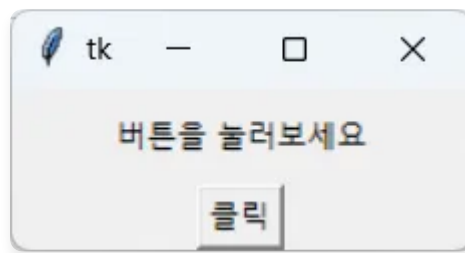
def hello():
    label.config(text="버튼을 클릭했습니다!")

root = tk.Tk()

label = tk.Label(root, text="버튼을 눌러보세요", padx=40, pady=10)
label.pack()

button = tk.Button(root, text="클릭", command=hello)
button.pack()

root.mainloop()
```



함수 정의와 호출

- 함수를 정의하고 버튼 클릭으로 실행하는 핵심 코드는 다음 부분입니다.

```
def hello():
    label.config(text="버튼을 클릭했습니다!")

button = tk.Button(root, text="클릭", command=hello)
```

- `text` 옵션은 버튼에 표시할 글자입니다.
- `command` 옵션은 버튼을 클릭했을 때 실행할 함수를 지정합니다.
- `config()` 는 이미 만들어진 위젯의 옵션을 변경할 때 사용합니다.
- 여기서는 라벨의 `text` 옵션을 새 문자열로 바꾸었습니다.

command 사용 시 주의할 점

- 버튼에서 가장 많이 실수하는 부분은 `command` 에 함수를 연결하는 방식입니다.
- 올바른 코드는 다음과 같습니다.

```
command=hello
```

- 그런데 다음과 같이 잘못된 코드를 사용하는 경우가 있습니다.

```
command=hello()
```

- 두 코드의 차이는 다음과 같습니다.

코드	의미
<code>command=hello</code>	버튼을 클릭했을 때 hello 함수 실행
<code>command=hello()</code>	프로그램 시작 시 hello()를 바로 실행

알아두기

함수보다 버튼을 먼저 만드는 경우

`command` 옵션에는 버튼을 클릭했을 때 실행할 함수를 지정합니다. 이때 함수 이름을 사용하려면, 버튼을 만드는 시점에 해당 함수가 이미 정의되어 있어야 합니다.

아래 코드는 hello 함수가 정의되기 전에 `command=hello`가 실행되므로 오류가 발생합니다.

```
button = tk.Button(root, text="클릭", command=hello)

def hello():
    print("Hello")
```

주요 옵션

옵션	허용 값	설명
<code>width</code>	0 이상의 정수 (텍스트는 글자 수, 이미지는 픽셀 단위)	버튼의 너비
<code>height</code>	0 이상의 정수 (텍스트는 줄 수, 이미지는 픽셀 단위)	버튼의 높이
<code>command</code>	함수 객체	버튼 클릭 시 실행할 함수(이벤트 핸들러) 지정
<code>state</code>	<code>"normal"</code> , <code>"disabled"</code> , <code>"active"</code> 중 하나	버튼의 상태 설정

Button 옵션과 상태 적용

- 다양한 옵션과 상태를 적용한 버튼의 예시 코드입니다.

```
import tkinter as tk

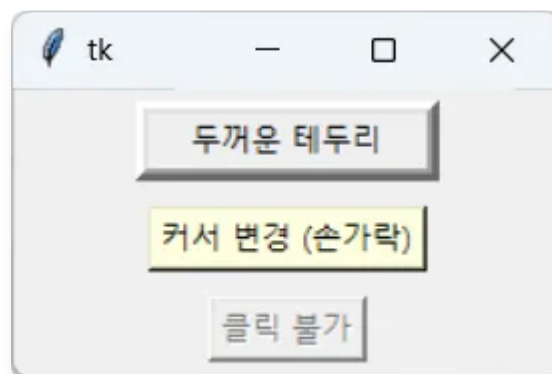
root = tk.Tk()

btn1 = tk.Button(root, text="두꺼운 테두리", bd=5, relief="raised", width=15)
btn1.pack(pady=5)

btn2 = tk.Button(root, text="커서 변경 (손가락)", cursor="hand2", bg="lightyellow")
btn2.pack(pady=5)

btn3 = tk.Button(root, text="클릭 불가", state="disabled")
btn3.pack(pady=5)

root.mainloop()
```



엔트리와 텍스트

tkinter에서 사용하는 **엔트리(Entry)** 와 **텍스트(Text)** 위젯을 살펴보겠습니다.

Entry 위젯은 한 줄짜리 짧은 문자열을 입력받을 때 사용합니다. Text 위젯은 여러 줄에 걸친 긴 문자열을 입력하거나 화면에 표시할 때 사용합니다.

이름, 아이디, 검색어처럼 짧고 단일한 값은 Entry를 사용하는 것이 적합합니다. 메모, 게시글, 설명문처럼 여러 줄로 입력될 수 있는 내용은 Text를 사용하는 것이 적합합니다.

- [Entry](#)
 - [Entry 사용하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
 - [비밀번호 입력 Entry 만들기](#)
- [Text](#)
 - [Text 사용하기](#)
 - [주요 메서드와 데이터 다루기](#)
 - [주요 옵션](#)

Entry

- Entry는 사용자가 한 줄의 텍스트를 입력할 수 있는 위젯으로 짧은 문자열을 입력받을 때 사용합니다.

Entry 사용하기

- 한 줄 입력 상자 Entry에 소문자를 입력하고 대문자로 변경하는 코드입니다.

```

import tkinter as tk

def change_text():
    text = entry.get()
    entry.delete(0, tk.END)
    entry.insert(0, text.upper())

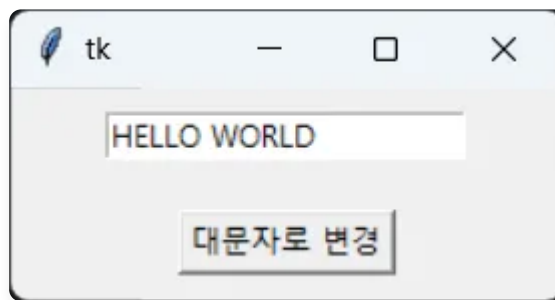
root = tk.Tk()

entry = tk.Entry(root)
entry.pack(padx=10, pady=10)

button = tk.Button(root, text="대문자로 변경", command=change_text)
button.pack(padx=10, pady=10)

root.mainloop()

```



주요 메서드

메서드	설명
<code>get()</code>	입력값 가져오기
<code>delete()</code>	입력값 지우기
<code>insert()</code>	입력값 넣기

📌 알아두기

메서드란?

tkinter에서 메서드(method)란 위젯이 가지고 있는 동작 기능을 말합니다. 옵션이 위젯의 모양이나 상태를 설정하는 값이라면, 메서드는 위젯에게 어떤 작업을 수행하도록 지시하는 명령입

니다.

주요 옵션

옵션	허용 값	설명
<code>width</code>	0 이상의 정수 (입력 문자 개수 기준)	너비를 입력 문자 개수 기준으로 설정
<code>font</code>	(<code>"맑은 고딕", 12</code>) 형식의 튜플 또는 문자열	입력 글자의 글꼴, 크기, 스타일을 설정
<code>fg</code>	색상 이름(<code>"black"</code>) 또는 16진수 (<code>"#000000"</code>)	입력 글자의 색상을 설정
<code>bg</code>	색상 이름(<code>"white"</code>) 또는 16진수 (<code>"#ffffff"</code>)	입력 상자의 배경색을 설정
<code>show</code>	단일 문자 (<code>"*"</code> 등)	입력한 문자를 다른 문자로 표시(주로 비밀번호 입력창)
<code>state</code>	<code>"normal"</code> , <code>"disabled"</code> , <code>"readonly"</code> 중 하나	입력 가능 상태를 설정

비밀번호 입력 Entry 만들기

- `show` 옵션을 사용하면 입력한 글자를 다른 문자로 가릴 수 있습니다.

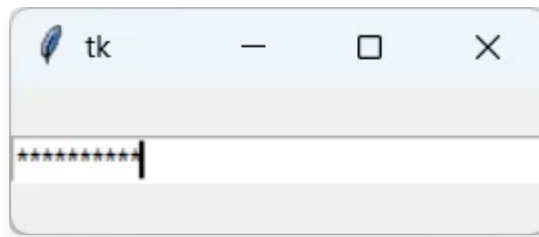
```
import tkinter as tk

root = tk.Tk()

password_entry = tk.Entry(root, show="*", width=30)
password_entry.pack(pady=20)

root.mainloop()
```

실행 후 비밀번호를 입력하면 실제 입력한 문자는 보이지 않고 * 문자로 표시됩니다.



Text

- Text는 사용자가 여러 줄의 텍스트를 입력할 수 있는 위젯입니다.
- 메모장, 게시글 작성창, 설명 입력창처럼 긴 글을 입력할 때 사용합니다.

Text 사용하기

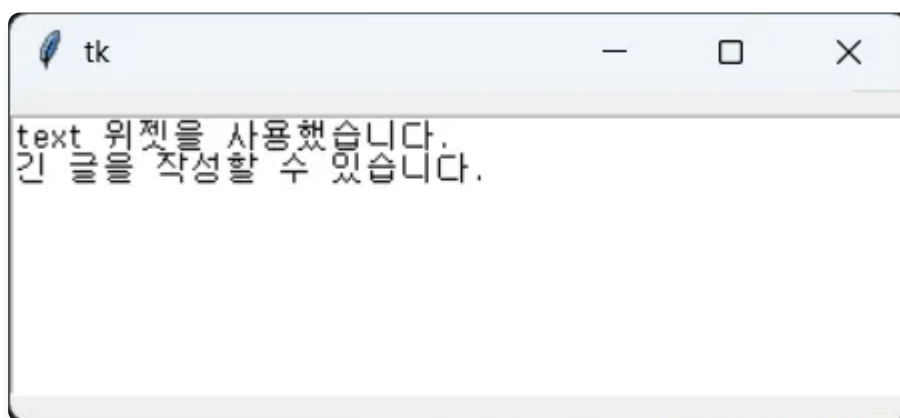
- 실행하면 여러 줄을 입력할 수 있는 큰 텍스트 영역이 나타납니다.

```
import tkinter as tk

root = tk.Tk()

text = tk.Text(root)
text.pack(pady=10)

root.mainloop()
```



주요 메서드와 데이터 다루기

메서드	설명	사용 예
<code>get()</code>	특정 범위의 텍스트 가져오기	<code>text.get("1.0", tk.END)</code>
<code>delete()</code>	특정 범위의 텍스트 지우기	<code>text.delete("1.0", tk.END)</code>
<code>insert()</code>	특정 위치에 텍스트 넣기	<code>text.insert("1.0", "안녕")</code>

- Text 위젯 역시 입력된 내용을 가져오거나(`get`), 지우거나(`delete`), 넣는(`insert`) 메서드를 사용합니다.
- 하지만 여러 줄을 다루기 때문에 Entry와는 값을 지정하는 인덱스(Index) 방식이 다릅니다.

알아두기

Text 위젯의 인덱스(Index)

Entry는 한 줄이므로 숫자 0부터 시작하지만, Text는 여러 줄을 다루므로 `"줄.칸"` 형태의 문자열 인덱스를 사용합니다.

- `"1.0"` : 1번째 줄의 0번째 칸(맨 처음)을 의미합니다.
- `tk.END` : 텍스트의 맨 끝 위치를 의미합니다.

따라서 Text 위젯에 적힌 모든 내용을 가져오거나 지우려면 항상 시작 위치를 0이 아닌 `"1.0"` 으로 지정해야 합니다.

주요 옵션

옵션	허용 값	설명
<code>width</code>	0 이상의 정수 (글자 수 기준)	위젯의 너비를 '글자 수' 기준으로 설정
<code>height</code>	0 이상의 정수 (줄 수 기준)	위젯의 높이를 '줄 수' 기준으로 설정
<code>font</code>	<code>("Arial", 12)</code> 형식의 튜플 또는 문자열	입력 글자의 글꼴, 크기 설정
<code>bg</code>	색상 이름(<code>"white"</code>) 또는 16진수 (<code>"#ffffff"</code>)	배경색을 설정
<code>fg</code>	색상 이름(<code>"black"</code>) 또는 16진수 (<code>"#000000"</code>)	글자색을 설정
<code>wrap</code>	<code>"word"</code> , <code>"char"</code> , <code>"none"</code> 중 하나	텍스트가 위젯 너비를 넘어갈 때 줄바꿈 방식 설정

체크, 라디오, 리스트

tkinter에서 사용하는 **Checkbox**, **Radiobutton**, **Listbox** 위젯을 살펴보겠습니다.

Checkbox은 여러 항목 중에서 필요한 항목을 선택하거나 해제할 때 사용합니다. Radiobutton은 여러 항목 중 하나만 선택할 때 사용합니다. Listbox는 여러 항목을 목록 형태로 보여주고, 사용자가 항목을 선택할 수 있게 할 때 사용합니다.

- [Checkbox](#)
 - [Checkbox 사용하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [Radiobutton](#)
 - [Radiobutton 사용하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [Listbox](#)
 - [Listbox 사용하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)

Checkbox

- Checkbox은 사용자가 항목을 선택하거나 선택 해제할 수 있는 위젯입니다.

Checkbox 사용하기

- 다음은 Checkbox을 사용하여 사용자의 선택 여부를 확인하는 코드입니다.


```

import tkinter as tk

def show_result():
    if agree_var.get():
        label.config(text="동의했습니다.")

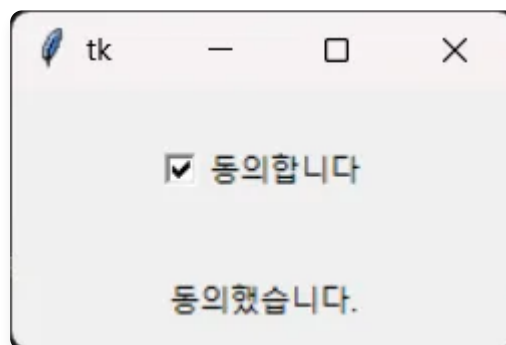
root = tk.Tk()
agree_var = tk.BooleanVar()

check = tk.Checkbutton(root, text="동의합니다", variable=agree_var, command=show_result)
check.pack(padx=20, pady=20)

label = tk.Label(root, text="")
label.pack(pady=10)

root.mainloop()

```



주요 메서드

메서드	설명
<code>select()</code>	체크 상태로 변경
<code>deselect()</code>	체크 해제 상태로 변경
<code>toggle()</code>	현재 상태를 반대로 변경
<code>invoke()</code>	사용자가 클릭한 것처럼 동작 실행

주요 옵션

옵션	허용 값	설명
<code>text</code>	임의의 문자열	체크 버튼 옆에 표시할 문자열 설정
<code>variable</code>	Tkinter 변수 객체	체크 상태를 저장할 변수 설정
<code>onvalue</code>	정수 <code>1</code> , 문자열 <code>"yes"</code> 등	체크되었을 때 변수에 저장할 값 설정
<code>offvalue</code>	정수 <code>0</code> , 문자열 <code>"no"</code> 등	체크 해제되었을 때 변수에 저장할 값 설정
<code>command</code>	함수 객체	체크 상태가 바뀔 때 실행할 함수 설정
<code>state</code>	<code>"normal"</code> , <code>"disabled"</code> 중 하나	위젯의 상태 설정

Radiobutton

- Radiobutton은 여러 항목 중에서 하나만 선택할 수 있는 위젯입니다.
- 같은 변수에 연결된 Radiobutton끼리는 하나의 그룹으로 동작합니다.

Radiobutton 사용하기

- 다음은 음식을 선택하는 Radiobutton 예제입니다.

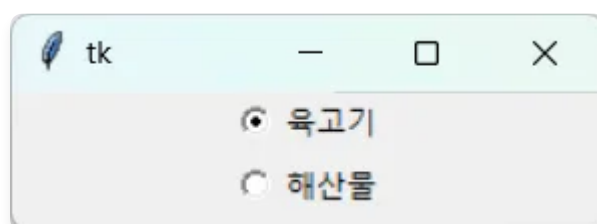
```
import tkinter as tk

root = tk.Tk()

food = tk.IntVar(value=1)

tk.Radiobutton(root, text="육고기", variable=food, value=1).pack()
tk.Radiobutton(root, text="해산물", variable=food, value=2).pack()

root.mainloop()
```



- Radiobutton은 `variable` 옵션과 `value` 옵션을 함께 사용합니다.
- 사용자가 특정 라디오 버튼을 선택하면, 해당 버튼의 `value` 값이 `variable` 에 저장됩니다.

주요 메서드

메서드	설명
<code>select()</code>	라디오 버튼을 선택
<code>deselect()</code>	라디오 버튼의 선택 해제
<code>invoke()</code>	사용자가 클릭한 것처럼 동작 실행

주요 옵션

옵션	허용 값	설명
<code>text</code>	임의의 문자열	라디오 버튼 옆에 표시할 문자열 설정
<code>variable</code>	Tkinter 변수 객체	선택값을 저장할 변수 설정
<code>value</code>	변수 타입에 맞는 값	라디오 버튼을 선택했을 때 변수에 저장할 값 설정
<code>command</code>	함수 객체	라디오 버튼을 선택했을 때 실행할 함수 설정
<code>state</code>	<code>"normal"</code> , <code>"disabled"</code> 중 하나	위젯의 상태 설정

📌 알아두기

Radiobutton은 같은 변수를 공유해야 합니다

여러 개의 Radiobutton이 하나의 그룹으로 동작하려면 모두 같은 `variable` 을 사용해야 합니다.

Listbox

- Listbox는 여러 항목을 목록 형태로 표시하고, 사용자가 항목을 선택할 수 있도록 하는 위젯입니다.

Listbox 사용하기

- 다음은 목록에서 항목을 선택하고, 선택한 항목을 화면에 표시하는 코드입니다.

```
import tkinter as tk

def show_selected(event):
    label.config(text="선택한 항목: " + listbox.get(listbox.curselection()))

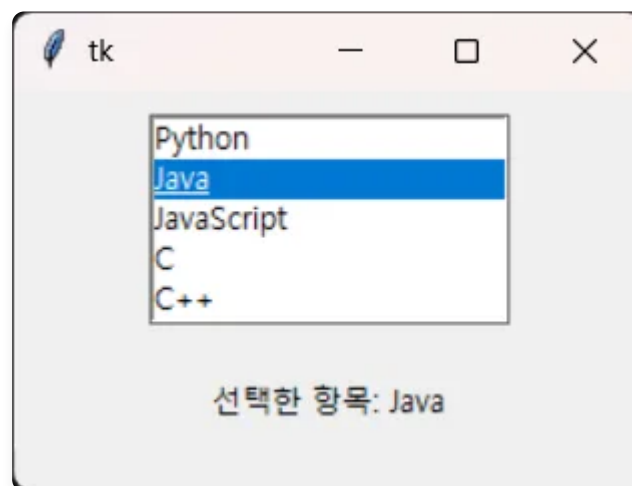
root = tk.Tk()
listbox = tk.Listbox(root, height=5)
listbox.pack(padx=10, pady=10)

for item in ["Python", "Java", "JavaScript", "C", "C++"]:
    listbox.insert(tk.END, item)

listbox.bind("<<ListboxSelect>>", show_selected)

label = tk.Label(root, text="")
label.pack(pady=10)

root.mainloop()
```



- `curselection()` 메서드는 현재 선택된 항목의 인덱스를 튜플 형태로 반환합니다.
- 선택된 항목이 있으면 `get()` 메서드를 사용해 해당 위치의 값을 가져올 수 있습니다.

주요 메서드

메서드	설명	사용 예
<code>insert()</code>	항목 추가	<code>listbox.insert(tk.END, "Python")</code>
<code>delete()</code>	항목 삭제	<code>listbox.delete(0)</code>
<code>get()</code>	항목 가져오기	<code>listbox.get(0)</code>
<code>curselection()</code>	선택된 항목의 인덱스 가져오기	<code>listbox.curselection()</code>
<code>size()</code>	전체 항목 개수 가져오기	<code>listbox.size()</code>
<code>selection_set()</code>	특정 항목 선택	<code>listbox.selection_set(0)</code>
<code>selection_clear()</code>	선택 해제	<code>listbox.selection_clear(0, tk.END)</code>

주요 옵션

옵션	허용 값	설명
<code>width</code>	0 이상의 정수 (글자 수 기준)	위젯의 너비를 글자 수 기준으로 설정
<code>height</code>	0 이상의 정수 (줄 수 기준)	표시할 항목의 줄 수 설정
<code>selectmode</code>	"browse", "single", "multiple", "extended" 중 하나	항목 선택 방식을 설정
<code>font</code>	("Arial", 12) 형식의 튜플 또는 문자열	목록 글자의 글꼴과 크기 설정
<code>fg</code>	색상 이름("black") 또는 16진수("#000000")	글자색 설정
<code>bg</code>	색상 이름("white") 또는 16진수("#ffffff")	배경색 설정

📌 알아두기

Listbox의 인덱스

Listbox의 항목 위치는 0부터 시작합니다. 첫 번째 항목은 0, 두 번째 항목은 1, 세 번째 항목은 2 입니다.

```
listbox.get(0)
```

위 코드는 Listbox의 첫 번째 항목을 가져옵니다.

스핀박스, 스케일

스핀박스(Spinbox) 와 스케일(Scale) 은 사용자로부터 특정 범위 내의 값을 입력받을 때 유용한 위젯입니다.

스핀박스는 위아래 화살표 버튼을 통해 정해진 범위의 값을 증감시키거나 직접 입력할 수 있으며, 스케일은 슬라이더를 드래그하여 연속적인 값을 직관적으로 선택할 수 있습니다.

두 위젯 모두 숫자 범위 제한, 증감 단위 설정 등의 기능을 제공하여 잘못된 입력을 방지하는 데 효과적입니다.

- [스핀박스](#)
 - [스핀박스 사용하기](#)
 - [문자열 목록을 사용하는 스핀박스](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [스케일](#)
 - [스케일 사용한 볼륨 값 선택](#)
 - [세로 방향 스케일 만들기](#)
 - [스케일 값이 바뀔 때 바로 실행하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)

스핀박스

- Spinbox는 입력 상자와 위아래 버튼이 함께 있는 위젯입니다.

스핀박스 사용하기

- 다음은 Spinbox를 사용하여 수량을 선택하는 예제입니다.

```

import tkinter as tk

def show_value():
    label.config(text="선택한 수량: " + spinbox.get())

root = tk.Tk()
root.title("Spinbox 예제")

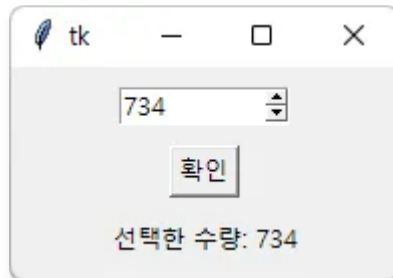
spinbox = tk.Spinbox(root, from_=1, to=10, width=10)
spinbox.pack(padx=10, pady=10)

button = tk.Button(root, text="확인", command=show_value)
button.pack(pady=5)

label = tk.Label(root, text="")
label.pack(pady=10)

root.mainloop()

```



- Spinbox는 Entry처럼 값을 직접 입력할 수 있고, 버튼을 눌러 값을 바꿀 수도 있습니다.

문자열 목록을 사용하는 스핀박스

- Spinbox는 숫자 범위뿐만 아니라 문자열 목록도 사용할 수 있습니다.
- 이때는 values 옵션을 사용합니다.


```

import tkinter as tk

def show_month():
    label.config(text="선택한 달: " + spinbox.get())

root = tk.Tk()
root.title("문자열 Spinbox")

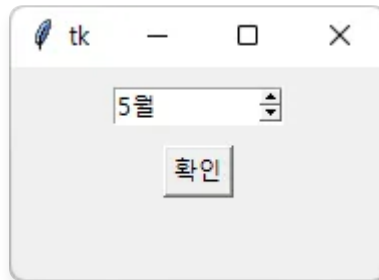
spinbox = tk.Spinbox(root, values=("1월", "2월", "3월", "4월", "5월"), width=10)
spinbox.pack(padx=10, pady=10)

button = tk.Button(root, text="확인", command=show_month)
button.pack(pady=5)

label = tk.Label(root, text="")
label.pack(pady=10)

root.mainloop()

```



주요 메서드

메서드	설명
<code>get()</code>	현재 값 가져오기
<code>delete()</code>	입력된 값 삭제
<code>insert()</code>	값 삽입

주요 옵션

옵션	허용 값	설명
<code>from_</code>	실수 또는 정수	시작값 설정
<code>to</code>	실수 또는 정수	끝값 설정
<code>increment</code>	0보다 큰 실수 또는 정수	값이 증가하거나 감소하는 단위 설정
<code>values</code>	문자열 튜플 또는 리스트	선택할 문자열 목록 설정
<code>width</code>	0 이상의 정수 (글자 수 기준)	위젯의 너비 설정
<code>command</code>	함수 객체	위아래 버튼을 눌러 값이 바뀔 때 실행할 함수 설정
<code>state</code>	<code>"normal"</code> , <code>"readonly"</code> , <code>"disabled"</code> 중 하나	입력 가능 상태 설정

■ 알아두기

`Spinbox` 는 사용자가 값을 직접 입력할 수 있다는 점에서 `Entry` 와 비슷합니다. 하지만 `Spinbox` 는 정해진 범위나 목록 안에서 값을 선택하도록 만들 수 있다는 차이가 있습니다.

예를 들어 단순히 이름을 입력받을 때는 `Entry` 가 적합하고, 수량이나 월처럼 제한된 값을 선택할 때는 `Spinbox` 가 적합합니다.

스케일

- `Scale` 은 슬라이더를 움직여 숫자 값을 선택할 수 있는 위젯입니다.
- 볼륨, 밝기, 크기, 속도처럼 일정한 범위 안에서 값을 조절할 때 자주 사용합니다.

스케일 사용한 볼륨 값 선택

- 다음은 `Scale` 을 사용하여 볼륨 값을 선택하는 예제입니다.

```
import tkinter as tk

def show_value():
    label.config(text="선택한 볼륨: " + str(scale.get()))

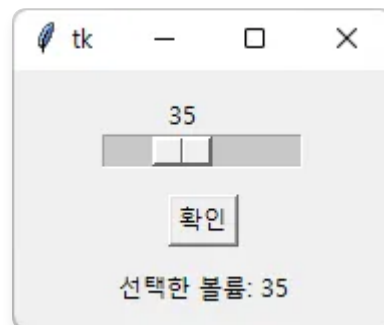
root = tk.Tk()

scale = tk.Scale(root, from_=0, to=100, orient="horizontal")
scale.pack(padx=10, pady=10)

button = tk.Button(root, text="확인", command=show_value)
button.pack(pady=5)

label = tk.Label(root, text="")
label.pack(pady=10)

root.mainloop()
```



- `Scale` 의 `get()` 메서드를 사용하면 현재 슬라이더의 값을 가져올 수 있습니다.
- `Scale` 에서 가져온 값은 숫자이므로 문자열과 연결할 때는 `str()` 을 사용하여 문자열로 바꾸어야 합니다.

세로 방향 스케일 만들기

- `Scale` 은 가로 방향뿐만 아니라 세로 방향으로도 만들 수 있습니다.
- 이때는 `orient` 옵션에 `"vertical"` 을 지정합니다.

```

import tkinter as tk

def show_value():
    label.config(text=f"선택한 밝기: {scale.get()}")

root = tk.Tk()

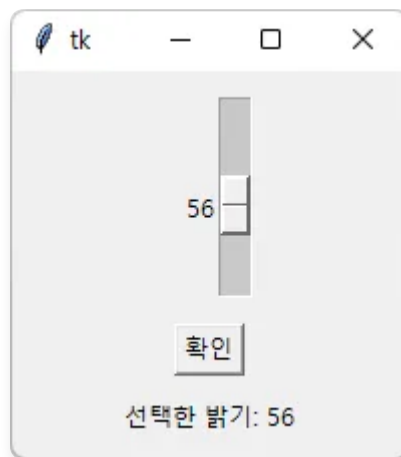
scale = tk.Scale(root, from_=0, to=100, orient="vertical")
scale.pack(pady=10)

tk.Button(root, text="확인", command=show_value).pack()

label = tk.Label(root)
label.pack(pady=10)

root.mainloop()

```



스케일 값이 바뀔 때 바로 실행하기

- `Scale` 은 버튼을 누르지 않아도 값이 바뀔 때마다 함수를 실행할 수 있습니다.
- 이때는 `command` 옵션을 사용합니다.

```
import tkinter as tk

def change_value(value):
    label.config(text=f"현재 값: {value}")

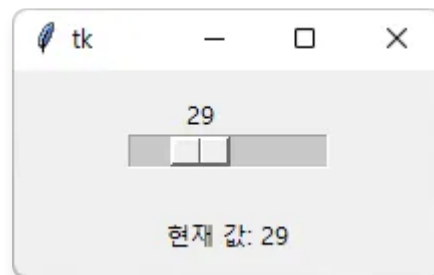
root = tk.Tk()

scale = tk.Scale(root, from_=0, to=100, orient="horizontal", command=change_value)
scale.pack(pady=10)

label = tk.Label(root, text="현재 값: 0")
label.pack(pady=10)

root.mainloop()
```

- `command` 에 연결된 함수는 `Scale` 의 값이 바뀔 때마다 실행됩니다.
- 이때 함수는 현재 `Scale` 값을 매개변수로 전달받습니다.
- 전달받는 값은 문자열 형태이므로 그대로 `Label` 에 표시할 수 있습니다.



주요 메서드

메서드	설명
<code>get()</code>	현재 Scale 값 가져오기
<code>set()</code>	Scale 값을 지정한 값으로 변경하기

주요 옵션

옵션	허용 값	설명
<code>from_</code>	실수 또는 정수	시작값 설정
<code>to</code>	실수 또는 정수	끝값 설정
<code>orient</code>	<code>"horizontal"</code> , <code>"vertical"</code> 중 하나	Scale 방향 설정
<code>length</code>	0 이상의 정수 (픽셀 단위)	Scale의 길이 설정
<code>resolution</code>	0보다 큰 실수 또는 정수	값이 움직이는 간격 설정
<code>tickinterval</code>	0 이상의 실수 또는 정수	눈금 표시 간격 설정
<code>label</code>	임의의 문자열	Scale 위젯에 표시할 제목 설정
<code>command</code>	함수 객체	값이 바뀔 때 실행할 함수 설정
<code>state</code>	<code>"normal"</code> , <code>"disabled"</code> 중 하나	사용 가능 상태 설정

알아두기

위젯마다 값을 가져오는 방식이 다릅니다

`Checkbutton` 과 `Radiobutton` 은 직접 위젯에서 값을 가져오기보다 연결된 변수에서 값을 가져오는 방식이 일반적입니다.

반면 `Listbox`, `Scale`, `Spinbox` 는 위젯의 `get()` 메서드를 사용하여 값을 가져올 수 있습니다.

단, `Listbox` 는 먼저 `curselection()` 으로 선택된 항목의 위치를 확인한 뒤, 그 위치를 이용해 값을 가져와야 합니다.

레이아웃 관리자

tkinter에서 위젯을 화면에 배치할 때는 레이아웃 관리자(geometry manager)를 사용합니다. 대표적으로 pack, grid, place 세 가지가 있으며, 상황에 맞는 관리자를 선택하는 것이 정돈된 인터페이스를 만드는 핵심입니다.

- [시작하기 전에](#)
 - [공통 개념](#)
 - [세 관리자 한눈에 보기](#)
- [pack\(\) 레이아웃](#)
 - [주요 옵션](#)
 - [side 예제](#)
 - [fill과 expand 예제](#)
 - [위젯 숨기기](#)
- [grid\(\) 레이아웃](#)
 - [주요 옵션](#)
 - [span과 sticky 예제](#)
 - [행·열 늘이기](#)
 - [위젯 숨기기](#)
- [place\(\) 레이아웃](#)
 - [주요 옵션](#)
- [레이아웃 관리자 혼용 규칙](#)
 - [Frame으로 계층 분리](#)

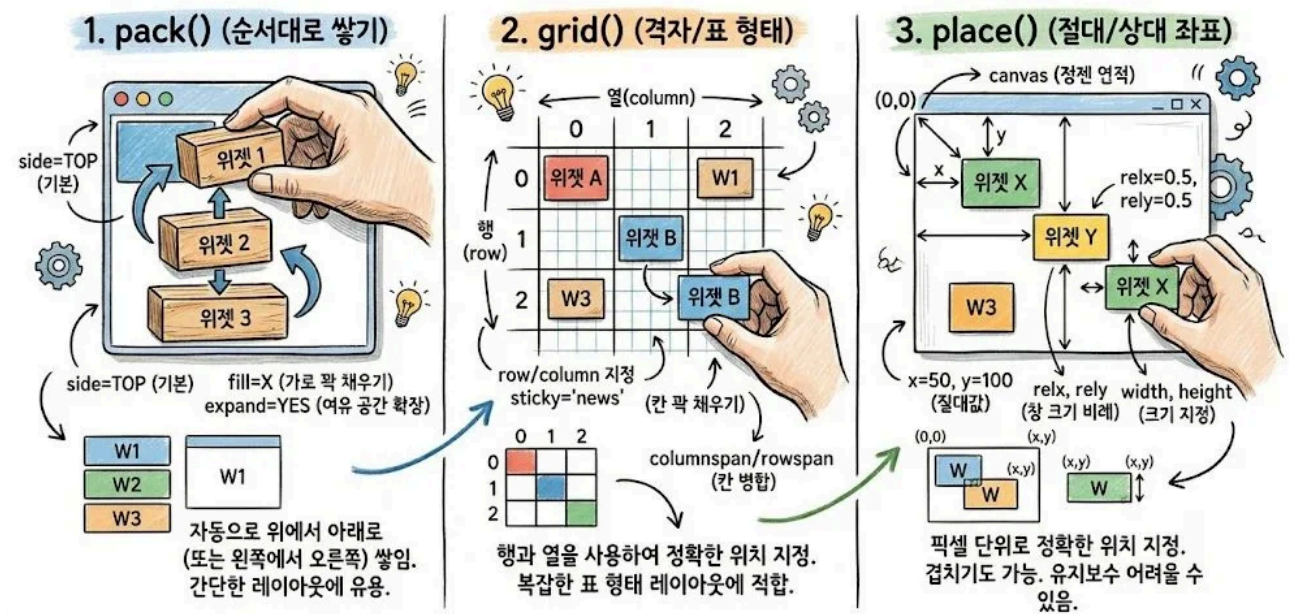
시작하기 전에

공통 개념

- 모든 위젯은 자신을 담는 부모 위젯 안에 배치됩니다. 보통 `root` 창이거나 `Frame` 입니다.
- 위젯을 생성하는 일(`tk.Label(...)`)과 화면에 배치하는 일(`.pack()` , `.grid()` , `.place()`)은 별개의 단계입니다. 배치 메서드를 호출하기 전까지 위젯은 화면에 나타나지 않습니다.

세 관리자 한눈에 보기

관리자	배치 방식	어울리는 상황
<code>pack()</code>	한 방향으로 차곡차곡 쌓음	단순한 세로/가로 나열
<code>grid()</code>	행·열 표 형태로 배치	입력 폼, 설정 화면
<code>place()</code>	좌표로 직접 고정	정밀한 고정 배치(특수한 경우)



pack() 레이아웃

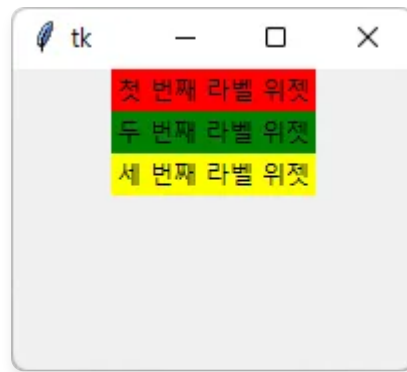
- `pack()` 은 위젯을 상자에 차곡차곡 넣듯 배치합니다. 기본 방향은 위에서 아래입니다.

```
import tkinter as tk

root = tk.Tk()
root.geometry("200x150")

tk.Label(root, text="첫 번째 라벨 위젯", bg="red").pack()
tk.Label(root, text="두 번째 라벨 위젯", bg="green").pack()
tk.Label(root, text="세 번째 라벨 위젯", bg="yellow").pack()

root.mainloop()
```

주요 옵션

옵션	허용 값	설명
<code>side</code>	<code>"top"</code> , <code>"bottom"</code> , <code>"left"</code> , <code>"right"</code>	위젯을 붙일 방향
<code>fill</code>	<code>"x"</code> , <code>"y"</code> , <code>"both"</code> , <code>"none"</code>	남는 공간을 채우는 방향
<code>expand</code>	<code>True</code> , <code>False</code>	남는 공간 차지 여부
<code>anchor</code>	<code>"n"</code> , <code>"s"</code> , <code>"e"</code> , <code>"w"</code> , <code>"ne"</code> , <code>"center"</code> 등	할당된 공간 안에서의 정렬
<code>padx</code> / <code>pady</code>	정수 또는 <code>(좌, 우)</code> / <code>(상, 하)</code> 튜플	위젯 바깥 여백
<code>ipadx</code> / <code>ipady</code>	정수	위젯 안쪽 여백

`side` 가 "공간을 어디서 떼어올지"를 정한다면, `anchor` 는 "그 공간 안에서 어느 쪽으로 정렬할지"를 정합니다.

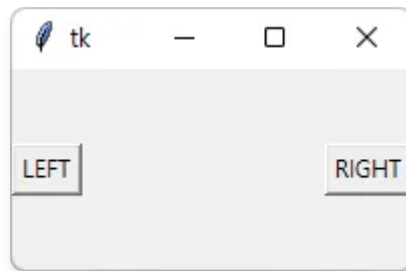
side 예제

```
import tkinter as tk

root = tk.Tk()
root.geometry("400x300")

tk.Button(root, text="LEFT").pack(side="left")
tk.Button(root, text="RIGHT").pack(side="right")

root.mainloop()
```



fill과 expand 예제

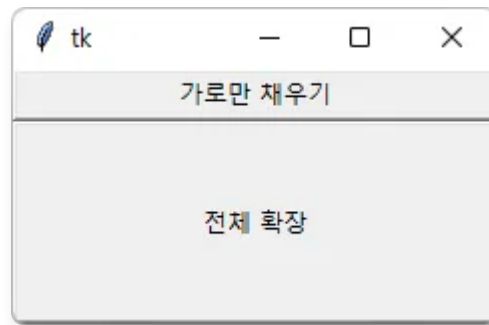
- `fill` 은 남는 공간을 채우는 방향, `expand=True` 는 남는 공간 자체를 차지하게 합니다. 둘을 함께 쓰면 창 크기에 맞게 위젯이 확장됩니다.

```
import tkinter as tk

root = tk.Tk()
root.geometry("250x120")

tk.Button(root, text="가로만 채우기").pack(fill="x")
tk.Button(root, text="전체 확장").pack(expand=True, fill="both")

root.mainloop()
```



위젯 숨기기

- 이미 배치한 위젯은 `pack_forget()` 으로 화면에서 떼어낼 수 있습니다. 객체는 그대로 남아 있어 나중에 다시 `pack()` 으로 배치할 수 있습니다.

```
btn.pack_forget()
btn.pack()
```

grid() 레이아웃

- `grid()` 는 화면을 **행(row)** 과 **열(column)** 로 나누어 배치합니다.
- 입력 폼처럼 표 형태의 UI에 가장 적합합니다. 번호는 0부터 시작합니다.

```
import tkinter as tk

root = tk.Tk()
root.geometry("180x50")

tk.Label(root, text="이름").grid(row=0, column=0)
tk.Entry(root).grid(row=0, column=1)
tk.Label(root, text="나이").grid(row=1, column=0)
tk.Entry(root).grid(row=1, column=1)

root.mainloop()
```



주요 옵션

옵션	허용 값	설명
<code>row</code>	0 이상의 정수	위젯이 놓일 행 번호(0부터)
<code>column</code>	0 이상의 정수	위젯이 놓일 열 번호(0부터)
<code>columnspan</code>	1 이상의 정수	위젯이 가로로 여러 칸 차지
<code>rowspan</code>	1 이상의 정수	위젯이 세로로 여러 칸 차지
<code>sticky</code>	"n", "s", "e", "w" 및 조합("nsew" 등)	셀 안에서 위젯을 붙일 방향

span과 sticky 예제

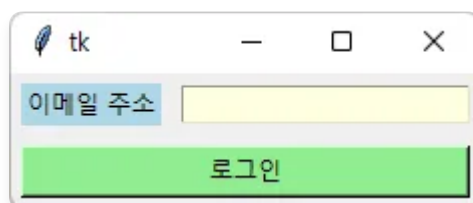
- `columnspan` 은 위젯을 여러 열에 걸쳐 늘리고, `sticky` 는 셀 안에서 위젯을 어느 방향에 붙일 지 정합니다.

```
import tkinter as tk

root = tk.Tk()
root.grid_columnconfigure(1, weight=1)      # 1번 열만 늘어남

opt = {"padx": 5, "pady": 5}
tk.Label(root, text="이 메 일 주 소", bg="lightblue").grid(row=1, column=0, sticky="w", **opt)
tk.Entry(root, bg="lightyellow").grid(row=1, column=1, sticky="we", **opt)
tk.Button(root, text="로 그 인", bg="lightgreen").grid(row=2, columnspan=2, sticky="we", **opt)

root.mainloop()
```



행.열 늘이기

- `sticky="we"` 만으로는 위젯이 늘어나지 않습니다.

- 창 크기에 맞춰 늘어나게 하려면 `grid_columnconfigure()` / `grid_rowconfigure()` 로 `weight` 를 줘야 합니다.
- `weight` 는 남은 공간을 나누는 비율입니다. 예를 들어 0번 열 `weight=1` , 1번 열 `weight=2` 면 1:2로 공간을 나눕니다.
- `weight=0` (기본값)인 행·열은 늘어나지 않습니다.

```
root.grid_rowconfigure(0, weight=1)
root.grid_columnconfigure(0, weight=1)
```

위젯 숨기기

메서드	설명
<code>grid_forget()</code>	위젯을 떼어내고 위치 정보도 잊음
<code>grid_remove()</code>	위젯을 떼어내되 위치 정보를 기억(다시 <code>grid()</code> 호출 시 원래 자리로)

■ 알아두기

`grid` 에서 빈 행·열은 자리를 차지하지 않습니다.

예를 들어 row 0과 row 5에만 위젯을 두면 중간이 비는 게 아니라 둘이 바로 붙습니다. 행 사이에 간격을 주고 싶다면 빈 행을 넣는 대신 `pady` 옵션을 사용하세요.

place() 레이아웃

- `place()` 는 위젯의 위치를 좌표로 직접 지정합니다.
- 직관적이지만 창 크기 변화에 약하므로, 꼭 필요한 고정 배치에만 제한적으로 사용합니다.

```
import tkinter as tk

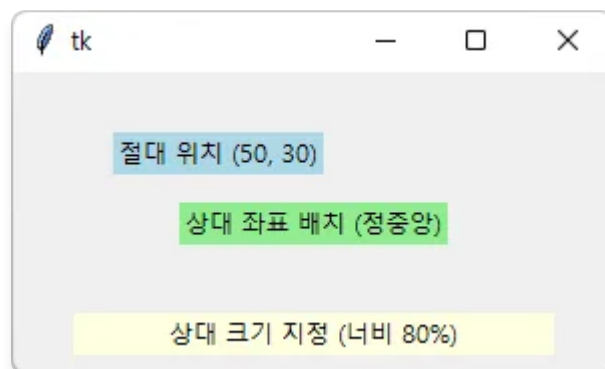
root = tk.Tk()
root.geometry("300x150")

tk.Label(root, text="절대 위치 (50, 30)", bg="lightblue").place(x=50, y=30) # 절대 좌표 배

tk.Label(root, text="상대 좌표 배치 (정중앙)", bg="lightgreen").place(
    relx=0.5, rely=0.5, anchor="center"
) # 상대 좌표 배치 (relx, rely는 0.0 ~ 1.0 비율)

tk.Label(root, text="상대 크기 지정 (너비 80%)", bg="lightyellow").place(
    relx=0.1, rely=0.8, relwidth=0.8
) # 상대 크기 지정 (relwidth, relheight)

root.mainloop()
```



주요 옵션

옵션	허용 값	설명
<code>x</code> , <code>y</code>	정수(픽셀)	절대 좌표(왼쪽 위 모서리 기준)
<code>relx</code> , <code>rely</code>	<code>0.0</code> ~ <code>1.0</code> 실수	부모 크기 대비 비율
<code>anchor</code>	<code>"n"</code> , <code>"s"</code> , <code>"e"</code> , <code>"w"</code> , <code>"ne"</code> , <code>"center"</code> 등	기준점 위치

⚠ `place()` 는 좌표로 고정하므로 다른 위젯과 자동으로 어울리지 않고, 창 크기나 해상도가 바뀌면 겹치거나 잘릴 수 있습니다. 일반적인 화면은 `pack()` 이나 `grid()` 로 구성하는 것이 좋습니다.

레이아웃 관리자 혼용 규칙

- 같은 부모 위젯 안에서 `pack()` 과 `grid()` 를 섞어 쓰면 안 됩니다.
- 두 관리자가 요구하는 창 크기가 충돌하면서 tkinter가 크기를 계속 재계산해, 창이 떨리거나 응답하지 않는 상태가 됩니다.
- 이때 명확한 예외(`TclError`)가 발생하지 않는 경우가 많아 원인을 찾기 어렵습니다.

Frame으로 계층 분리

- `Frame` 으로 구역을 나누면 됩니다.
- 핵심은 "같은 부모 안에서만 한 가지 관리자를 쓴다" 는 것입니다. 부모가 다르면(창 vs. 프레임) 서로 다른 관리자를 써도 괜찮습니다.

```
import tkinter as tk

root = tk.Tk()
root.geometry("360x150")
root.configure(bg="#797777")

# 위쪽 프레임: 입력 폼 (내부는 grid)
top = tk.Frame(root, bg="#d7ecff", padx=5, pady=5)
top.pack(side="top", fill="x", padx=5, pady=5)

tk.Label(top, text="이 메 일 주 소", bg="#d7ecff").grid(row=0, column=0, sticky="w", padx=(0, 10))
tk.Entry(top, bg="white").grid(row=0, column=1, sticky="we")
top.grid_columnconfigure(1, weight=1) # 입력칸이 가로로 늘어나도록

#아래쪽 프레임: 버튼 (내부는 pack)
bottom = tk.Frame(root, bg="#ffe0e0", padx=5, pady=5)
bottom.pack(side="bottom", fill="x", padx=5, pady=5)

tk.Button(bottom, text="확 인", bg="#7bc96f", fg="white", width=8).pack(side="left")
tk.Button(bottom, text="취 소", bg="#e87a7a", fg="white", width=8).pack(side="right")

root.mainloop()
```



이벤트와 콜백

지금까지 '라벨', '버튼', '엔트리' 같은 위젯을 만들고 배치하며 화면을 구성하는 법을 배웠습니다. 하지만 "사용자가 버튼을 눌렀을 때 무슨 일이 일어나는가?" 라는 질문에는 아직 답하지 못했습니다.

이 장에서는 GUI 프로그램의 심장이라 할 수 있는 **이벤트(event)**와 **콜백(callback)**의 동작 원리를 파헤칩니다. 정적인 화면이 어떻게 살아 움직이는 프로그램이 되는지, 그 메커니즘을 체계적으로 살펴봅시다.

- [이벤트 기반 프로그래밍의 이해](#)
 - [이벤트 기반 패러다임](#)
 - [이벤트](#)
 - [메인 루프의 역할](#)
 - [콜백](#)
- [command 옵션을 통한 콜백 등록](#)
 - [기본적인 콜백](#)
 - [함수 "객체"를 넘긴다는 것](#)
 - [command의 한계](#)
- [bind\(\)를 통한 이벤트 바인딩](#)
 - [command보다 넓은 세계](#)
 - [키보드 이벤트\(Key Event\) 바인딩과 콜백 함수 활용](#)
 - [코드 한 줄씩 살펴보기](#)
 - [콜백의 시그니처](#)
- [이벤트 시퀀스와 객체의 속성](#)
 - [주요 이벤트 시퀀스](#)
 - [이벤트 객체의 속성](#)

이벤트 기반 프로그래밍의 이해

이벤트 기반 패러다임

- 순차 실행 방식 프로그램은 코드가 위에서 아래로 실행되고 끝나면 종료됩니다.

- 하지만 GUI 프로그램은 다릅니다. "사용자가 무언가 할 때까지 기다리는" 방식으로 동작합니다.
- 즉, 프로그램의 주도권이 사용자에게 있다는 것이 핵심입니다.

이벤트

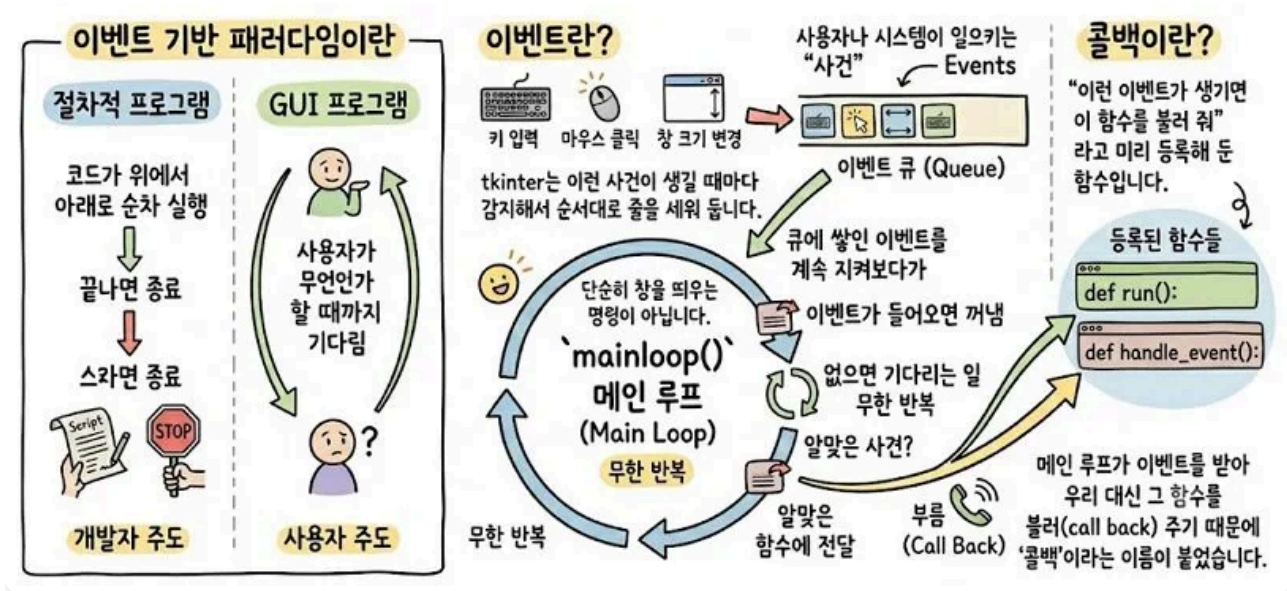
- 이벤트는 키 입력, 마우스 클릭, 창 크기 변경처럼 사용자나 시스템이 일으키는 "사건" 입니다.
- tkinter는 이런 사건이 생길 때마다 감지해서 순서대로 줄을 세워 둡니다. 이 줄을 큐(queue)라고 합니다.

메인 루프의 역할

- `mainloop()` 는 단순히 창을 띄우는 명령이 아닙니다.
- 큐에 쌓인 이벤트를 계속 지켜보다가, 이벤트가 들어오면 꺼내서 알맞은 함수에 전달하고, 없으면 기다리는 일을 무한 반복합니다.

콜백

- 콜백은 "이런 이벤트가 생기면 이 함수를 불러 줘" 라고 미리 등록해 둔 함수입니다.
- 메인 루프가 이벤트를 받아 우리 대신 그 함수를 불러(call back) 주기 때문에 '콜백'이라는 이름이 붙었습니다.



command 옵션을 통한 콜백 등록

기본적인 콜백

- `Button`, `Checkbutton`, `Radiobutton`, `Scale` 등 다수의 위젯은 `command` 옵션을 제공합니다.

- 여기에 함수 객체를 전달하면 위젯이 활성화될 때 그 함수가 호출됩니다.

앞서 강좌에서 `command` 옵션을 통한 콜백 등록은 여러 코드에서 이미 사용되었습니다.

함수 "객체"를 넘긴다는 것

- 초보자가 가장 흔히 저지르는 실수는 `command=on_click()` 처럼 괄호를 붙이는 것입니다.
- 우리가 원하는 것은 함수를 실행하는 것이 아니라 "나중에 실행할 함수 자체"를 넘기는 것이므로 괄호 없이 이름만 전달해야 합니다.

command의 한계

- `command` 로 등록한 콜백은 호출될 때 tkinter가 인자를 자동으로 넘겨주지 않습니다.
- 만약 콜백에 내가 정한 값을 넘기고 싶다면 `lambda` 로 감싸야 됩니다. (예 : `command=lambda: on_click(item_id)`)
- 다만 클릭 좌표나 눌린 키처럼 이벤트 자체에 대한 정보는 `command` 로는 얻을 수 없습니다. 이런 정보가 필요하다면 다음에 설명할 `bind()` 를 사용해야 합니다.

bind()를 통한 이벤트 바인딩

command보다 넓은 세계

- `bind()` 는 거의 모든 위젯에서 사용할 수 있습니다.
- `command` 가 다루지 못하는 마우스 이동, 키보드 입력, 포커스 변화 등 훨씬 다양한 이벤트를 처리할 수 있습니다.

키보드 이벤트(Key Event) 바인딩과 콜백 함수 활용

- 입력창에 타이핑을 할 때마다 방금 누른 키가 무엇인지 라벨에 즉각적으로 표시해주는 코드입니다.

```

import tkinter as tk

def on_key(event):
    result_label.config(text=f"입력된 키 : {event.char}")

root = tk.Tk()

entry = tk.Entry(root, width=30)
entry.pack(padx=20, pady=10)

result_label = tk.Label(root, text="입력된 키 : ", bg="lightyellow")
result_label.pack(padx=20, pady=10)

entry.bind("<Key>", on_key)

root.mainloop()

```



코드 한 줄씩 살펴보기

- 키 입력 이벤트를 처리할 콜백 함수

```

def on_key(event):
    result_label.config(text=f"입력된 키 : {event.char}")

```

이벤트가 발생했을 때 실행될 동작을 미리 정의해 두는 곳입니다.

- `bind()` 메서드를 활용한 이벤트 연결

```
entry.bind("<Key>", on_key)
```

만들어둔 함수를 실제 위젯(엔트리)의 특정 이벤트(키보드 입력)와 엮어주는 핵심 코드입니다.

콜백의 시그니처

- `bind()` 로 등록하는 콜백은 반드시 하나의 인자(관례적으로 event)를 받아야 합니다.
- `command` 방식과 가장 큰 차이점이며, 깜빡하면 `TypeError`가 발생합니다.

이벤트 시퀀스와 객체의 속성

주요 이벤트 시퀀스

- 마우스 클릭, 키보드 입력 등 사용자가 취하는 특정 행동(이벤트)을 식별하기 위해 Tkinter에서 사용하는 규격화된 문자열 코드입니다.
- `<Button-1>` 이나 `<KeyPress>` 처럼 꺾쇠괄호로 묶인 형태를 띠며, 어떤 이벤트에 반응할지 지정하는 역할을 합니다.

분류	시퀀스	설명
마우스	<Button-1>	좌클릭
마우스	<Button-2>	휠 클릭
마우스	<Button-3>	우클릭
마우스	<ButtonRelease-1>	버튼 땀
마우스	<B1-Motion>	좌클릭 드래그
마우스	<Double-Button-1>	더블클릭
마우스	<Enter> / <Leave>	위젯 영역 진입 / 이탈
키보드	<KeyPress>	모든 키
키보드	<KeyPress-Return>	엔터
키보드	<Key-Escape>	ESC
키보드	<Control-c>	Ctrl+C 조합
키보드	<a>	일반 문자키 (간단 표기)
윈도/포커스	<Configure>	크기·위치 변경
윈도/포커스	<FocusIn> / <FocusOut>	포커스 획득 / 상실
윈도/포커스	<Destroy>	위젯 소멸

시퀀스는 <수식어들-종류-상세> 구조이므로 <Shift-Up> , <Control-Alt-Delete> 같은 복합 시퀀스도 표현할 수 있습니다.

이벤트 객체의 속성

- 이벤트가 발생했을 때 생성되는 이벤트 객체(event) 안에 담겨 있는 세부 데이터들을 의미합니다.

속성	설명
<code>event.x</code> , <code>event.y</code>	마우스 좌표 (위젯 기준)
<code>event.x_root</code> , <code>event.y_root</code>	마우스 좌표 (화면 기준)
<code>event.char</code>	실제 입력 문자
<code>event.keysym</code>	키 이름
<code>event.widget</code>	이벤트가 발생한 위젯
<code>event.num</code>	마우스 버튼 번호 (1=좌, 2=휠, 3=우)

`event.widget` 을 활용하면 여러 위젯에 같은 콜백을 바인딩한 뒤 콜백 안에서 발생 주체를 구분할 수 있습니다.

📌 알아두기

`char` 와 `keysym` , 언제 무엇을 쓸까?

문자키(a, 1, 한글 등)는 `event.char` 로 입력된 글자를 얻습니다. 하지만 Enter, Shift, 방향키, F1 같은 특수키는 `event.char` 가 비어 있을 수 있으니 `event.keysym` (예: Return, Up, Shift_L)으로 식별하세요.

메뉴 바 구성

이 장에서는 tkinter의 **메뉴(Menu)**에 대해 알아보겠습니다.

- [메뉴](#)
 - [메뉴 만드는 순서](#)
 - [메뉴 사용](#)
 - [메뉴 계층 구조 한눈에 보기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [ContextMenu](#)
 - [ContextMenu 구현](#)

메뉴

- Menu는 프로그램 창 상단에 보이는 메뉴바입니다. 메모장에서 보이는 파일 / 편집 / 보기 같은 것들이 바로 Menu입니다.
- 큰 메뉴바 안에 작은 메뉴(파일, 편집 등)를 넣고, 그 안에 실제 항목(열기, 저장 등)을 넣는 계층 구조를 갖습니다.

메뉴 만드는 순서

- 메뉴바 역할을 할 Menu바를 생성합니다.
- 그 안에 넣을 하위 메뉴(Menu)를 생성합니다.
- 하위 메뉴는 add_command 등으로 항목을 채웁니다.
- add_cascade로 하위 메뉴를 메뉴바에 연결합니다.
- root.config(menu=...)로 메뉴바를 창에 부착합니다.

메뉴 사용


```

import tkinter as tk

root = tk.Tk()
root.geometry("400x120")

menubar = tk.Menu(root)    # 1) 메뉴바 생성

file_menu = tk.Menu(menubar, tearoff=0)    # 2) 하위 메뉴 생성
file_menu.add_command(label="열기", command=lambda: print("열기 클릭"))    # 3) 항목 채우기
file_menu.add_separator()
file_menu.add_command(label="종료", command=root.destroy)

menubar.add_cascade(label="파일", menu=file_menu)    # 4) 메뉴바에 연결

root.config(menu=menubar)    # 5) 창에 부착
root.mainloop()

```



메뉴 계층 구조 한눈에 보기

- 앞서 코드를 그림으로 나타내면 다음과 같습니다.
- add_cascade로 연결한 "파일"이 메뉴바 아래에 매달리고, 그 안에 항목들이 차례로 들어간 모습입니다.

```

menubar (root에 부착)
└─ "파일" (cascade)
    ├── 열기 (command)
    ├── ─── (separator)
    └─ 종료 (command)

```

주요 메서드

메서드	설명
<code>add_command(label, command)</code>	클릭하면 동작하는 일반 항목 추가
<code>add_cascade(label, menu)</code>	하위 메뉴를 펼치는 항목 추가 (메뉴바에 메뉴 연결)
<code>add_separator()</code>	항목 사이에 구분선 추가
<code>add_checkbutton(label, variable)</code>	켜고 끌 수 있는 체크 항목 추가
<code>add_radiobutton(label, variable, value)</code>	여러 개 중 하나만 선택하는 항목 추가
<code>tk_popup(x, y)</code>	지정한 화면 좌표에 메뉴를 팝업으로 표시
<code>delete(index)</code>	항목 삭제

주요 옵션

옵션	허용 값	설명
<code>tearoff</code>	<code>0</code> , <code>1</code> (또는 <code>False</code> / <code>True</code>)	메뉴 상단 점선(분리 기능) 표시 여부
<code>label</code>	문자열	화면에 표시되는 항목 이름
<code>command</code>	함수(콜러블)	항목 클릭 시 실행할 함수
<code>accelerator</code>	문자열(예: <code>"Ctrl+S"</code>)	단축키 안내 문자열
<code>state</code>	<code>tk.NORMAL</code> , <code>tk.DISABLED</code> , <code>tk.ACTIVE</code>	항목 상태
<code>underline</code>	0 이상의 정수(글자 인덱스)	항목 이름 중 한 글자에 밑줄을 그어 키보드 접근 문자로 지정

⚠ `tearoff` 는 기본값이 켜져 있습니다. Menu를 만들 때 `tearoff` 를 지정하지 않으면 기본값이 `1` 이라, 메뉴 맨 위에 점선이 하나 생깁니다.

ContextMenu

- ContextMenu(컨텍스트 메뉴)는 마우스 오른쪽 버튼을 클릭했을 때 나타나는 팝업 메뉴입니다.
- 그런데 tkinter에는 ContextMenu라는 별도 클래스가 없습니다
- 그래서 일반 Menu 위젯을 만들고, 우클릭 이벤트와 연결해 필요할 때 `tk_popup()` 메서드로 띄웁니다

ContextMenu 구현

```

import tkinter as tk

root = tk.Tk()
root.geometry("300x120")

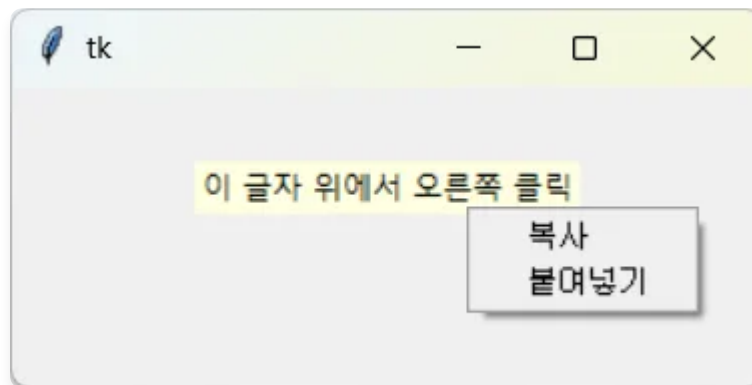
menu = tk.Menu(root, tearoff=0)
menu.add_command(label="복사", command=lambda: print("복사"))
menu.add_command(label="붙여넣기", command=lambda: print("붙여넣기"))

label = tk.Label(root, text="이 글자 위에서 오른쪽 클릭", bg="lightyellow")
label.pack(pady=30)

label.bind("<Button-3>", lambda e: menu.tk_popup(e.x_root, e.y_root))

root.mainloop()

```



■ 알아두기

팝업 좌표는 왜 `x_root` 일까?

`tk_popup()` 은 화면 전체를 기준으로 한 절대 좌표를 받습니다. 그래서 위젯 기준인 `e.x`, `e.y` 가 아니라 화면 기준인 `e.x_root`, `e.y_root` 를 넘겨야 클릭한 자리에 정확히 메뉴가 뜹니다.

대화상자 (Dialog)

이 장에서는 **대화상자(Dialog)**에 대해 알아보겠습니다. 대화상자는 사용자와 상호작용하기 위해 뜨는 팝업창으로, 알림을 띄우거나 사용자의 선택·입력을 받을 때 사용합니다.

- [대화상자](#)
 - [표준 대화상자](#)
 - [사용자 정의 대화상자](#)
- [표준 대화상자](#)
 - [세 가지 주요 모듈](#)
 - [대화상자는 "값을 반환한다"](#)
 - [messagebox](#)
 - [filedialog](#)
 - [simpledialog](#)
- [사용자 정의 대화상자](#)
 - [Toplevel이란?](#)
 - [비 모달 방식 대화상자](#)
 - [모달\(Modal\) 대화상자](#)

대화상자

- Dialogs는 사용자와 잠깐 대화하기 위해 뜨는 팝업창입니다.
- tkinter는 자주 쓰는 대화상자를 미리 만들어 제공하므로, 창을 직접 디자인하지 않고 함수 한 줄로 호출할 수 있습니다.
- 대화상자는 크게 두 종류로 나뉩니다.

표준 대화상자

- tkinter가 제공하는 `messagebox`, `filedialog`, `simpledialog`가 있습니다.

사용자 정의 대화상자

- `Toplevel`로 대화상자로 입력 필드가 여러 개거나 디자인을 짜야 할 때 직접 만드는 창입니다.

표준 대화상자

세 가지 주요 모듈

- 표준 대화상자는 용도에 따라 세 모듈로 나뉩니다.

모듈	용도	주요 메서드
<code>messagebox</code>	알림, 경고, 예/아니오 질문	<code>showinfo</code> , <code>askyesno</code>
<code>filedialog</code>	파일/폴더 열기·저장 창	<code>askopenfilename</code> , <code>asksaveasfilename</code>
<code>simpdialog</code>	짧은 텍스트·숫자 입력받기	<code>askstring</code> , <code>askinteger</code>

- 세 모듈 모두 tkinter 안에 있지만, 자동으로 따라오지 않으므로 별도로 `import` 해야 합니다.

```
from tkinter import messagebox, filedialog, simpdialog
```

대화상자는 "값을 반환한다"

- 표준 대화상자의 중요한 특징은 사용자의 선택·입력 결과를 반환값으로 돌려준다는 점입니다.
- 따라서 보통 `결과 = 대화상자함수(...)` 형태로 받아서 사용합니다.

모듈	반환값	취소했을 때
<code>messagebox</code>	<code>True</code> / <code>False</code>	보통 <code>False</code>
<code>filedialog</code>	파일 경로 문자열	빈 문자열 <code>""</code>
<code>simpdialog</code>	입력한 텍스트/숫자	<code>None</code>

⚠ 사용자 취소하면 정상적인 값 대신 `None`이나 빈 문자열이 돌아옵니다. 반환값을 그대로 쓰기 전에 반드시 확인하는 습관이 필요합니다.

messagebox

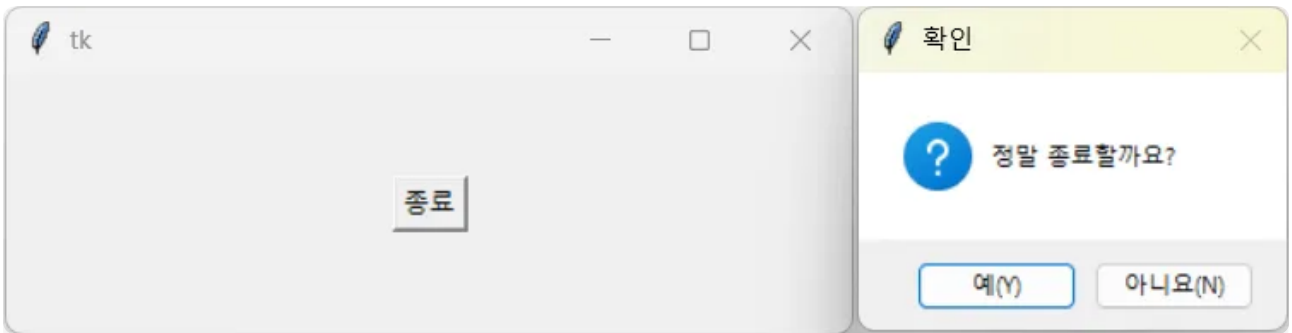
- 가장 자주 쓰는 모듈로 정보를 보여주거나 예/아니오를 묻습니다.

```
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()

def exit_app():
    if messagebox.askyesno("확인", "정말 종료할까요?"):
        root.destroy()

tk.Button(root, text="종료", command=exit_app).pack(padx=180, pady=48)
root.mainloop()
```



- messagebox 주요 메서드

메서드	설명
<code>showinfo</code>	정보 알림 표시 (확인 버튼)
<code>showwarning</code>	경고 알림 표시 (확인 버튼)
<code>showerror</code>	오류 알림 표시 (확인 버튼)
<code>askquestion</code>	예/아니오 질문, <code>"yes"</code> / <code>"no"</code> 문자열 반환
<code>askyesno</code>	예/아니오 질문, <code>True</code> / <code>False</code> 반환
<code>askokcancel</code>	확인/취소 질문, <code>True</code> / <code>False</code> 반환
<code>askretrycancel</code>	재시도/취소 질문, <code>True</code> / <code>False</code> 반환
<code>askyesnocancel</code>	예/아니오/취소 질문, <code>True</code> / <code>False</code> / <code>None</code> 반환

filedialog

- 파일이나 폴더의 경로를 사용자에게 받아옵니다.

```
from tkinter import filedialog

path = filedialog.askopenfilename(
    title="파일 선택",
    filetypes=[("텍스트 파일", "*.txt"), ("모든 파일", "*.*")]
)
if path:
    print("선택한 파일:", path)
```

- filedialog 주요 메서드

메서드	설명
<code>askopenfilename</code>	파일 하나 열기. 선택한 경로 문자열 반환 (취소 시 <code>""</code>)
<code>askopenfilenames</code>	여러 파일 열기. 경로 튜플 반환 (취소 시 빈 튜플)
<code>asksaveasfilename</code>	저장할 경로 받기. 경로 문자열 반환 (취소 시 <code>""</code>)
<code>askdirectory</code>	폴더 선택. 폴더 경로 문자열 반환 (취소 시 <code>""</code>)
<code>askopenfile</code>	파일을 열어 읽기용 파일 객체 반환 (취소 시 <code>None</code>)
<code>asksaveasfile</code>	파일을 열어 쓰기용 파일 객체 반환 (취소 시 <code>None</code>)

- filedialog 주요 옵션

옵션	허용 값	설명
<code>title</code>	문자열	대화상자 제목
<code>initialdir</code>	폴더 경로 문자열	처음 열릴 폴더 경로
<code>filetypes</code>	(설명, 패턴) 튜플의 리스트 (예: <code>[("텍스트", "*.txt")]</code>)	표시할 파일 종류 제한
<code>defaulttextension</code>	확장자 문자열 (예: <code>".txt"</code>)	저장 시 자동으로 붙는 기본 확장자
<code>initialfile</code>	파일명 문자열	저장 대화상자에 미리 채워질 파일명

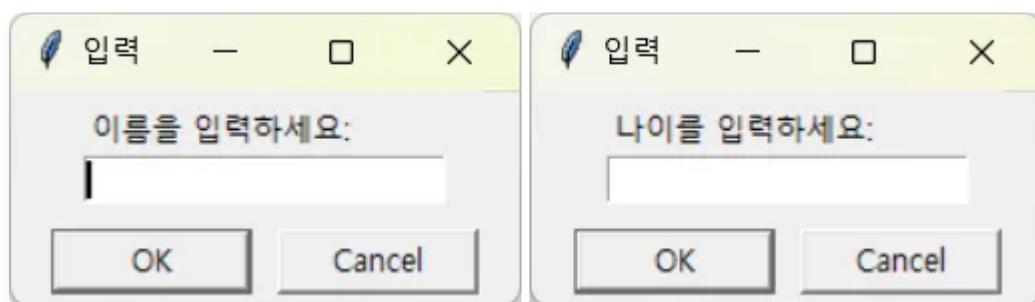
simplifiedialog

- 짧은 텍스트나 숫자를 입력받을 때 사용합니다.

```
from tkinter import simplifiedialog

name = simplifiedialog.askstring("입력", "이름을 입력하세요.")
age = simplifiedialog.askinteger("입력", "나이를 입력하세요.", minvalue=0, maxvalue=120)

if name:
    print(f"{name}님, {age}세")
```



- simplifiedialog 주요 메서드

메서드	설명
<code>askstring</code>	문자열 입력받기. 입력한 문자열 반환 (취소 시 <code>None</code>)
<code>askinteger</code>	정수 입력받기. <code>minvalue</code> , <code>maxvalue</code> 로 범위 제한 가능 (취소 시 <code>None</code>)
<code>askfloat</code>	실수 입력받기. <code>minvalue</code> , <code>maxvalue</code> 로 범위 제한 가능 (취소 시 <code>None</code>)

사용자 정의 대화상자

- `messagebox` 나 `filedialog` 는 정해진 모양만 제공합니다.
- 입력 필드가 여러 개 필요하거나 디자인을 직접 짜고 싶다면 새 창을 직접 만들어야 하며, 이때 사용하는 것이 `Toplevel` 위젯입니다.

Toplevel이란?

- `Tk()` 가 프로그램의 메인 창이라면, `Toplevel()` 은 그 위에 추가로 띄우는 새 창입니다.
 - 메인 창처럼 `Label` , `Entry` , `Button` 등 모든 위젯을 넣을 수 있습니다.
 - 메인 창에 종속된 자식 창이므로, 메인 창을 닫으면 함께 닫힙니다.

비 모달 방식 대화상자

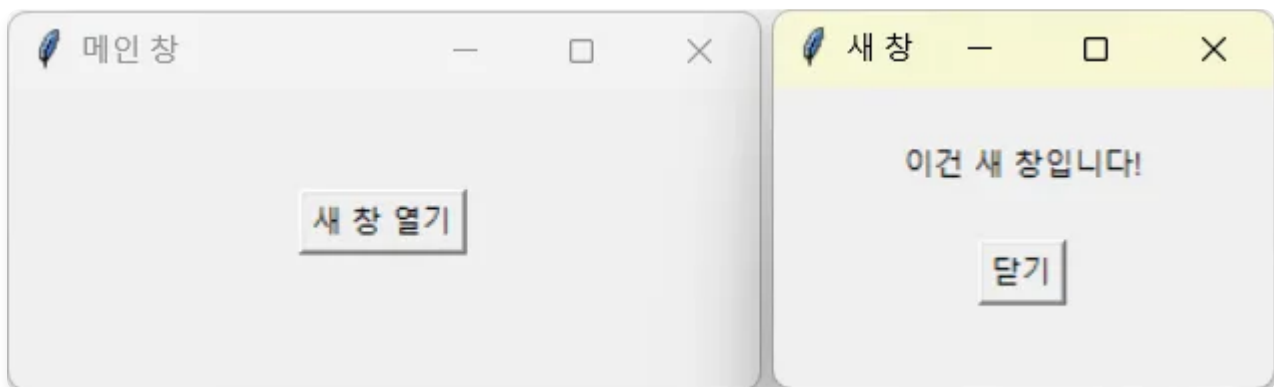
- 메인 창과 동시에 떠 있으며 둘 다 자유롭게 조작할 수 있는 방식을 비 모달(modeless) 이라고 합니다.

```
import tkinter as tk

root = tk.Tk()
root.title("메인 창")
root.geometry("300x200")

def open_window():
    win = tk.Toplevel(root)          # 새 창 생성
    win.title("새 창")
    tk.Label(win, text="이건 새 창입니다!").pack(pady=20)
    tk.Button(win, text="닫기", command=win.destroy).pack()

tk.Button(root, text="새 창 열기", command=open_window).pack(pady=50)
root.mainloop()
```



모달(Modal) 대화상자

- 사용자가 입력을 끝낼 때까지 메인 창을 막아두는 방식을 모달(Modal) 창이라 합니다.
- 설정 변경이나 필수 입력처럼 반드시 응답을 받아야 할 때 사용합니다.
- 모달은 다음 세 가지 설정으로 만듭니다.

메서드	설명
<code>transient(root)</code>	메인 창에 종속시킴 (항상 위에 표시)
<code>grab_set()</code>	모든 입력을 이 창에만 집중
<code>wait_window(dialog)</code>	이 창이 닫힐 때까지 코드 실행을 멈추고 대기

```

import tkinter as tk

root = tk.Tk()
root.title("메인 창")
root.geometry("300x100")

result = tk.Label(root, text="결과 : 없음")
result.pack(pady=10)

def ask_name():
    dialog = tk.Toplevel(root)
    dialog.title("이름 입력")

    tk.Label(dialog, text="이름을 입력해 주세요!").pack(pady=5)
    entry = tk.Entry(dialog)
    entry.pack(padx=20)

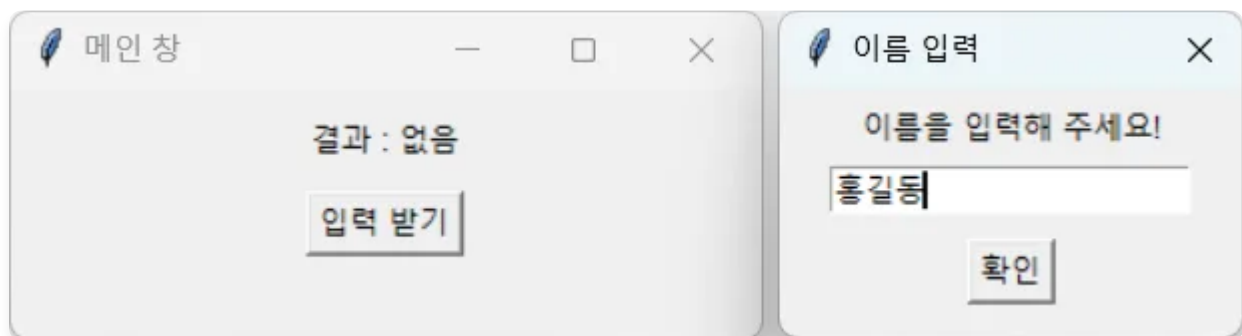
    def confirm():
        result.config(text=f"결과 : {entry.get()}")
        dialog.destroy()

    tk.Button(dialog, text="확인", command=confirm).pack(pady=10)

    dialog.transient(root)
    dialog.grab_set()
    root.wait_window(dialog)

tk.Button(root, text="입력 받기", command=ask_name).pack()
root.mainloop()

```



캔버스 (Canvas)

Canvas는 tkinter에서 선, 다각형, 원과 같은 복잡한 도형을 그리고, 텍스트나 위젯, 이미지 등을 화면에 자유롭게 배치할 수 있게 해주는 도화지 같은 위젯입니다.

- [도형 그리기 \(Shapes\)](#)
 - [선, 사각형, 원 그리기](#)
 - [선, 사각형, 점선 원 그리기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [텍스트 및 이미지 삽입](#)
 - [캔버스에 텍스트와 사진 출력하기](#)
 - [문구와 이미지 출력하기](#)
 - [주요 메서드](#)
- [객체 제어 및 애니메이션](#)
 - [도형 이동 및 속성 변경하기](#)
 - [버튼 클릭으로 원을 오른쪽으로 이동시키기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [이벤트 처리 \(마우스 드로잉\)](#)
 - [마우스 드래그 따라 선 그리기](#)

도형 그리기 (Shapes)

선, 사각형, 원 그리기

- Canvas는 다양한 도형을 그리는 create_ 로 시작하는 메서드를 제공합니다.
- 각 도형은 고유한 ID(정수)를 반환하며, 이를 통해 나중에 도형을 수정할 수 있습니다.
- 원(Oval)은 해당 원을 둘러싸는 가상의 직사각형(Bounding Box)의 좌측 상단 좌표와 우측 하단 좌표를 입력하여 그립니다.

선, 사각형, 점선 원 그리기

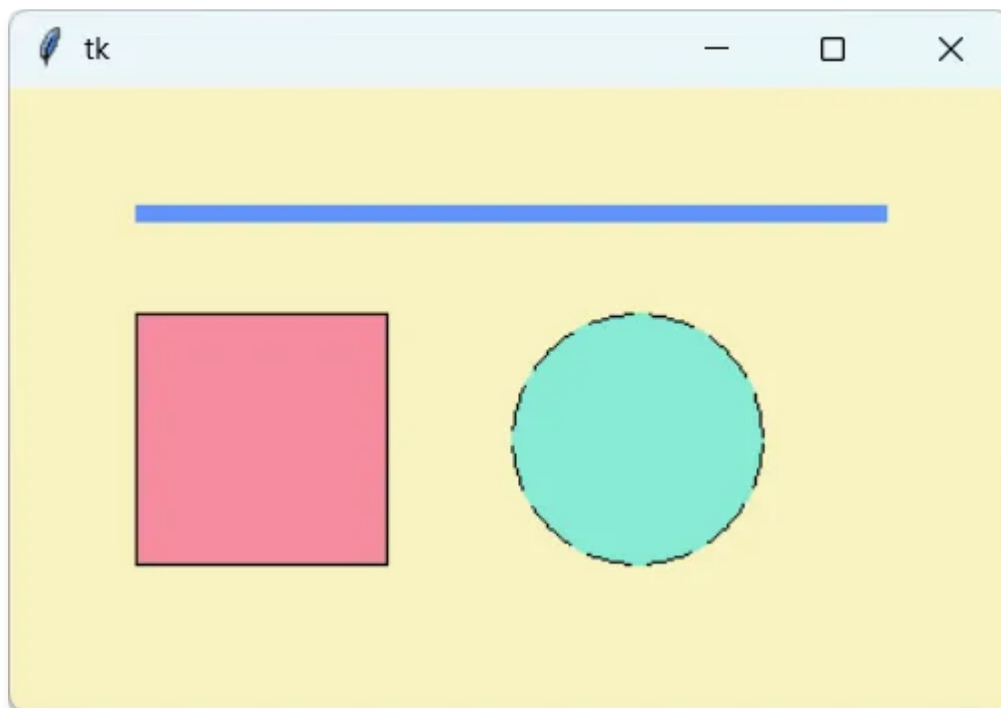
- 캔버스에 직선과 사각형, 점선 테두리의 타원을 각각 색을 달리하여 출력합니다.

```
import tkinter as tk

root = tk.Tk()
canvas = tk.Canvas(root, width=400, height=250, bg="#F8F3C0", highlightthickness=0)
canvas.pack()

canvas.create_line(50, 50, 350, 50, fill="#6194FA", width=7)
canvas.create_rectangle(50, 90, 150, 190, fill="#F58C9F", width=1)
canvas.create_oval(200, 90, 300, 190, fill="#88EBD4", width=1, dash=(5, 2))

root.mainloop()
```



주요 메서드

메서드	설명
<code>create_line(x1, y1, x2, y2, ...)</code>	직선 그리기
<code>create_rectangle(x1, y1, x2, y2, ...)</code>	직사각형 그리기
<code>create_oval(x1, y1, x2, y2, ...)</code>	타원이나 원 그리기
<code>create_polygon(x1, y1, x2, y2, x3, y3, ...)</code>	3개 이상의 점을 연결한 다각형 그리기

주요 옵션

옵션	허용 값	설명
<code>fill</code>	색상 이름(<code>"red"</code>) 또는 16진수 값 (<code>"#ff0000"</code>)	도형 내부를 채울 색상
<code>outline</code>	색상 이름(<code>"black"</code>) 또는 16진수 값 (<code>"#000000"</code>)	도형의 테두리 색상
<code>width</code>	정수 또는 실수 (픽셀 단위, 예: <code>2</code> , <code>1.5</code>)	선이나 테두리의 두께
<code>dash</code>	정수 튜플(예: <code>(5, 3)</code>) 또는 문자열(예: <code>".."</code> , <code>"-"</code>)	점선 스타일 지정 (예 : (5, 3)은 5픽셀 굵고 3픽셀 띄움)

텍스트 및 이미지 삽입

캔버스에 텍스트와 사진 출력하기

- Canvas 상의 특정 좌표를 지정하여 문자열을 렌더링할 수 있습니다.
- `PhotoImage` 객체를 활용하여 이미지를 삽입할 수 있습니다.
- 이미지는 가비지 컬렉터(Garbage Collector)에 의해 삭제되지 않도록 전역 변수로 관리하거나 객체에 참조를 남겨야 합니다.

문구와 이미지 출력하기

- 캔버스 중앙에 텍스트를 표시하고, 이미지 파일이 있으면 출력하며 없으면 안내 문구로 대체합니다.
- 코드가 저장된 동일한 경로에 200x200 크기의 'image.png' 파일을 위치해 둡니다.

```

import tkinter as tk

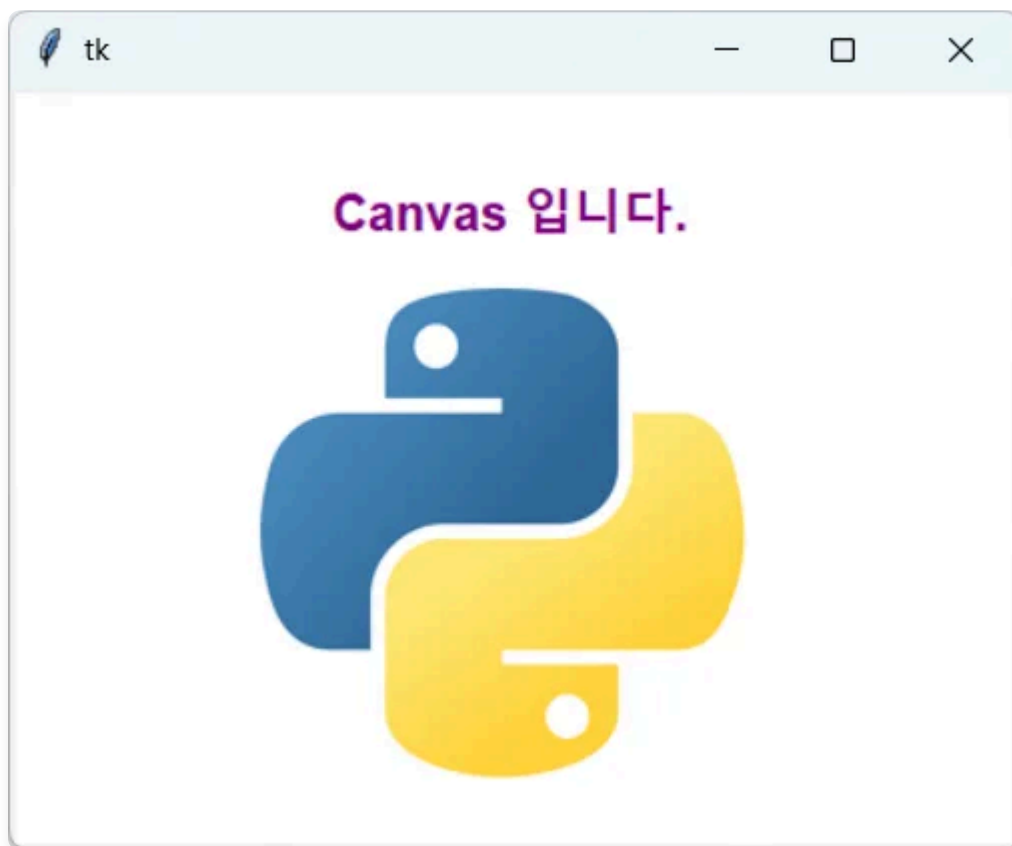
root = tk.Tk()
canvas = tk.Canvas(root, width=400, height=300, bg="white")
canvas.pack()

canvas.create_text(200, 50, text="Canvas 입니다.", font=("Arial", 15, "bold"), fill="purple")

try:
    img = tk.PhotoImage(file="image.png")
    canvas.image = img
    canvas.create_image(200, 180, image=img, anchor="center")
except tk.TclError:
    canvas.create_text(200, 180, text="(이미지가 없습니다.)")

root.mainloop()

```



주요 메서드

메서드	설명
<code>create_text(x, y, ...)</code>	지정한 좌표에 텍스트를 출력
<code>create_image(x, y, ...)</code>	지정한 좌표에 이미지를 출력

객체 제어 및 애니메이션

도형 이동 및 속성 변경하기

- Canvas에 그려진 모든 항목은 고유 식별자(ID) 또는 사용자가 지정한 문자열 태그(Tag)로 제어할 수 있습니다.
- `move()` 메서드와 `after()` 메서드를 결합하면 간단한 애니메이션 효과를 구현할 수 있습니다.

버튼 클릭으로 원을 오른쪽으로 이동시키기

- 버튼을 누르면 파란 원이 50ms 간격으로 오른쪽으로 움직이다가 화면 끝에 닿으면 멈춥니다.

```
import tkinter as tk

def move_circle():
    canvas.move("my_circle", 5, 0)
    coords = canvas.coords("my_circle")

    if coords[2] < 400:
        root.after(50, move_circle)

root = tk.Tk()
canvas = tk.Canvas(root, width=400, height=200, bg="white")
canvas.pack()

canvas.create_oval(10, 80, 50, 120, fill="blue", tags="my_circle")

btn = tk.Button(root, text="이동 시작", command=move_circle)
btn.pack(pady=10)

root.mainloop()
```



주요 메서드

메서드	설명
<code>move(id_or_tag, dx, dy)</code>	객체를 현재 위치에서 dx, dy만큼 이동
<code>coords(id_or_tag, [x1, y1...])</code>	객체의 현재 좌표를 반환하거나 새로운 좌표로 재설정
<code>itemconfig(id_or_tag, options)</code>	생성된 객체의 옵션(색상, 두께 등)을 동적으로 변경
<code>delete(id_or_tag)</code>	객체를 캔버스에서 삭제함. 모두 지우려면 <code>canvas.delete("all")</code> 을 사용

주요 옵션

옵션	허용 값	설명
<code>tags</code>	문자열 또는 문자열 튜플 (예 : ("tag1", "tag2"))	도형에 하나 이상의 태그를 부여하여, 동일 태그가 지정된 여러 도형을 그룹으로 묶어 한 번에 조작(이동, 색상 변경, 삭제 등)할 수 있게 함

이벤트 처리 (마우스 드로잉)

- Canvas는 사용자의 마우스 클릭, 드래그, 키보드 입력 등의 이벤트를 바인딩하여 반응형 프로그램을 만들 수 있습니다.

마우스 드래그 따라 선 그리기

- 마우스 왼쪽 버튼을 누른 채 움직이면 그 궤적을 따라 검은 선이 이어져 그림판처럼 그림을 그립니다.

```
import tkinter as tk

old_x, old_y = None, None

def start_draw(event):
    global old_x, old_y
    old_x, old_y = event.x, event.y

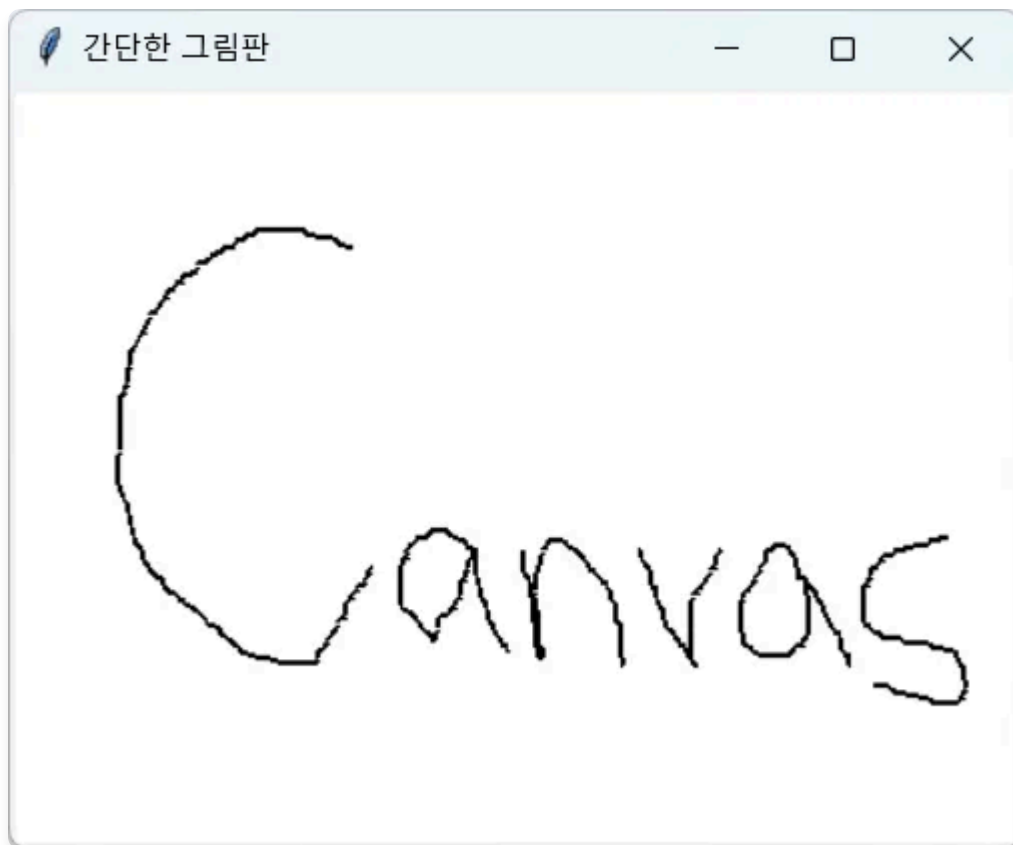
def draw_line(event):
    global old_x, old_y
    if old_x and old_y:
        canvas.create_line(old_x, old_y, event.x, event.y, width=2, fill="black")
        old_x, old_y = event.x, event.y

root = tk.Tk()

canvas = tk.Canvas(root, width=400, height=300, bg="white", cursor="crosshair")
canvas.pack()

canvas.bind("<Button-1>", start_draw)
canvas.bind("<B1-Motion>", draw_line)

root.mainloop()
```



ttk 위젯과 스타일링

ttk는 Tk 8.5부터 도입된 테마 적용 위젯 모음으로 tkinter.ttk 모듈을 통해 사용합니다. 기존 tkinter 위젯보다 현대적인 외관을 제공하며, 스타일과 동작을 분리한 구조가 특징입니다.

- [ttk 소개와 기본 개념](#)
 - [tkinter 위젯과 ttk 위젯의 차이](#)
 - [제공 위젯](#)
 - [기본 사용 예제](#)
- [스타일링과 테마](#)
 - [Style을 이용한 외관 제어](#)
 - [코드 한 줄씩 살펴보기](#)
 - [테마 선택](#)
 - [코드 한 줄씩 살펴보기](#)
 - [상태\(state\)와 동적 스타일](#)

ttk 소개와 기본 개념

tkinter 위젯과 ttk 위젯의 차이

- 외관이 OS 테마(Windows, macOS, Linux)에 자연스럽게 어울립니다.
- 색상·폰트 등을 위젯 옵션이 아닌 Style 객체로 관리합니다. 그래서 bg=, fg= 같은 옵션이 ttk 위젯에는 없거나 무시되는 경우가 많습니다.
- ttk는 Tk 8.5부터 도입되었습니다. Tk 버전은 다음과 같이 확인할 수 있습니다.

```
import tkinter
print("Tk Version:", tkinter.TkVersion)
print("Tcl Version:", tkinter.TclVersion)
```

제공 위젯

- ttk가 제공하는 주요 위젯은 다음과 같습니다.

Button, Checkbutton, Radiobutton, Label, Entry,
Frame, LabelFrame, Menubutton, Scale, Scrollbar,
Combobox, Notebook, Progressbar, Separator,
Sizegrip, Treeview, Spinbox, PanedWindow

기본 사용 예제

- ttk가 제공하는 `Label`, `Entry`, `Button` 위젯을 사용한 예제 코드입니다.

```
import tkinter as tk
from tkinter import ttk

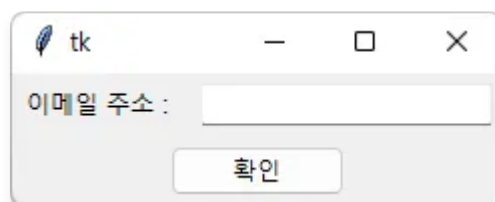
root = tk.Tk()

label = ttk.Label(root, text="이름 : ")
label.grid(row=0, column=0, padx=5, pady=5)

entry = ttk.Entry(root)
entry.grid(row=0, column=1, padx=5, pady=5)

button = ttk.Button(root, text="확인", command=lambda: print(entry.get()))
button.grid(row=1, column=0, columnspan=2, pady=5)

root.mainloop()
```



스타일링과 테마

Style을 이용한 외관 제어

- ttk의 핵심은 `ttk.Style` 객체입니다.
- 위젯 클래스마다 스타일 이름(예: `TButton`, `TLabel`)이 정해져 있으며, 이를 통해 외관을 일괄 변경할 수 있습니다.

- 일반 tkinter 위젯은 위젯마다 `font`, `bg` 등의 옵션을 직접 지정하지만, `ttk`는 스타일을 한 번 정의해두면 같은 스타일을 쓰는 모든 위젯에 일괄 적용됩니다.

```
import tkinter as tk
from tkinter import ttk

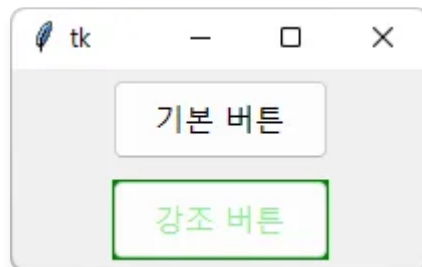
root = tk.Tk()

style = ttk.Style()
style.configure("TButton", font=("맑은 고딕", 11), padding=6)
style.configure("Accent.TButton", foreground="lightgreen", background="green")

btn1 = ttk.Button(root, text="기본 버튼")
btn1.pack(padx=50, pady=5)

btn2 = ttk.Button(root, text="강조 버튼", style="Accent.TButton")
btn2.pack(padx=50, pady=5)

root.mainloop()
```



코드 한 줄씩 살펴보기

- 스타일을 제어하는 객체를 생성

```
style = ttk.Style()
```

이 객체를 통해 모든 `ttk` 위젯의 외관을 설정합니다.

- 기본 버튼 스타일인 `TButton` 을 수정

```
style.configure("TButton", font=("맑은 고딕", 11), padding=6)
```

별도의 스타일을 지정하지 않은 모든 `ttk.Button` 에 이 설정(글꼴, 안쪽 여백)이 자동으로 적용됩니다.

- Accent.TButton이라는 사용자 정의 스타일 생성

```
style.configure("Accent.TButton", foreground="lightgreen", background="green")
```

TButton을 기반으로 하므로 위에서 지정한 글꼴과 여백은 그대로 유지되고, 글자색과 배경색만 추가로 지정됩니다.

- 기본 스타일 적용

```
btn1 = ttk.Button(root, text="기본 버튼")
```

style 옵션을 생략했으므로 기본 스타일인 TButton이 적용됩니다.

- 사용자 정의 스타일을 적용

```
btn2 = ttk.Button(root, text="강조 버튼", style="Accent.TButton")
```

`style="Accent.TButton"` 을 지정하여 앞서 정의한 사용자 정의 스타일을 적용합니다.

- 사용자 정의 스타일 이름 규칙은 `접두어.기본스타일` 형태로, 마지막에 기본 스타일 이름(TButton 등)을 붙여야 ttk가 어떤 위젯에 적용할 스타일인지 올바르게 인식합니다.

테마 선택

- ttk는 테마를 제공하며 다음 코드로 사용 가능한 테마와 현재 사용중인 테마를 확인할 수 있습니다.

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()

style = ttk.Style()
print(style.theme_names())
print(style.theme_use())

root.mainloop()
```

사용 가능한 테마는 운영체제(Windows, Linux, mac)마다 다릅니다.

- 테마를 변경하는 코드입니다.

```

import tkinter as tk
from tkinter import ttk

root = tk.Tk()
style = ttk.Style()

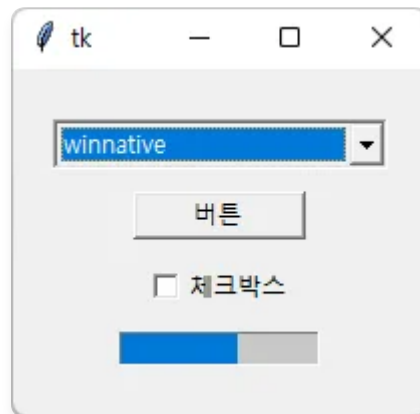
frm = ttk.Frame(root, padding=20)
frm.pack()

cur = tk.StringVar(value=style.theme_use())
cb = ttk.Combobox(frm, values=style.theme_names(), textvariable=cur, state="readonly")
cb.pack(pady=5)
cb.bind("<<ComboboxSelected>>", lambda e: style.theme_use(cur.get()))

ttk.Button(frm, text="버튼").pack(pady=5)
ttk.Checkbutton(frm, text="체크박스").pack(pady=5)
ttk.Progressbar(frm, value=60).pack(pady=5)

root.mainloop()

```



코드 한 줄씩 살펴보기

- 콤보박스에 연결할 문자열 변수 정의

```
cur = tk.StringVar(value=style.theme_use())
```

초기값을 현재 적용된 테마 이름으로 설정합니다. `style.theme_use()` 를 인자 없이 호출하면 지금 쓰이는 테마를 반환합니다.

- 선택 가능한 테마 목록을 콤보박스로

```
cb = ttk.Combobox(frm, values=style.theme_names(), textvariable=cur, state="readonly")
```

`style.theme_names()` 가 플랫폼별 테마 이름 튜플을 돌려주며 사용자가 목록에서 고르게 합니다.

- 항목 선택 시 이벤트에 함수를 연결

```
cb.bind("<<ComboboxSelected>>", lambda e: style.theme_use(cur.get()))
```

콤보박스에서 선택된 테마 이름 `cur.get()` 을 `style.theme_use()` 에 넘겨 테마를 교체하면, 모든 ttk 위젯이 새 테마로 다시 그려집니다.

상태(state)와 동적 스타일

- ttk 위젯은 `active`, `pressed`, `disabled`, `focus` 등의 상태를 가지며, `style.map()` 으로 상태별 외관을 지정할 수 있습니다.

```

import tkinter as tk
from tkinter import ttk

root = tk.Tk()

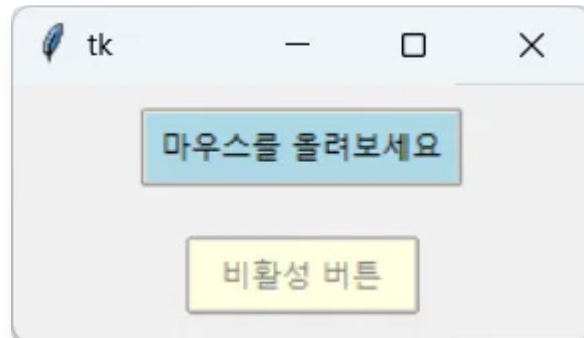
style = ttk.Style()
style.theme_use("clam")

style.map("TButton",
        background=[("active", "lightblue"),
                    ("pressed", "blue"),
                    ("disabled", "lightyellow")],
        foreground=[("disabled", "gray")])

ttk.Button(root, text="마우스를 올려보세요").pack(padx=20, pady=10)
ttk.Button(root, text="비활성 버튼", state="disabled").pack(padx=20, pady=10)

root.mainloop()

```



ttk 고유 위젯

ttk는 Combobox, Progressbar, Notebook, Treeview 등 기존 tkinter에 없던 고유의 위젯을 지원하여 사용할 수 있습니다.

- [콤보박스](#)
 - [콤보박스 사용하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [프로그레스바](#)
 - [프로그레스바 사용하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [노트북 \(탭\)](#)
 - [노트북 으로 탭 기능 사용하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [트리뷰](#)
 - [트리뷰 사용하기](#)
 - [주요 메서드](#)
 - [주요 옵션](#)

콤보박스

- 사용자가 드롭다운 목록에서 항목을 고르거나, 직접 값을 입력할 수 있습니다.
- 항목이 선택되면 `<<ComboboxSelected>>` 가상 이벤트가 발생합니다.

콤보박스 사용하기

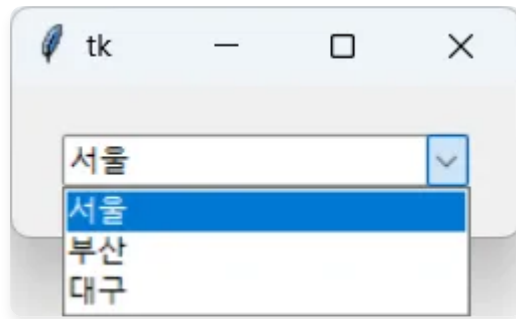
- 리스트로 넘겨받은 도시 이름을 콤보박스에서 사용해 선택된 값을 출력합니다.

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()

combo = ttk.Combobox(root, values=["서울", "부산", "대구"])
combo.set("서울")
combo.bind("<<ComboboxSelected>>", lambda e: print(combo.get()))
combo.pack(padx=20, pady=20)

root.mainloop()
```



주요 메서드

메서드	설명
<code>get()</code>	현재 입력되거나 선택된 값을 반환
<code>set(value)</code>	콤보박스의 값을 지정한 값으로 설정
<code>current(index)</code>	인자가 없으면 선택된 항목의 인덱스를 반환하고, 인덱스를 주면 해당 항목을 선택
<code>insert(index, value)</code>	입력 영역의 지정 위치에 문자열을 삽입
<code>delete(first, last)</code>	입력 영역에서 지정 범위의 문자를 삭제
<code>configure(options)</code>	위젯의 옵션을 동적으로 변경 (<code>config</code> 도 동일)
<code>bind(event, func)</code>	특정 이벤트에 콜백 함수를 연결

주요 옵션

옵션	허용 값	설명
<code>values</code>	리스트 또는 튜플 (예: <code>['A', 'B', 'C']</code>)	드롭다운 목록에 표시할 항목
<code>state</code>	<code>normal</code> , <code>readonly</code> , <code>disabled</code>	입력 상태를 지정
<code>textvariable</code>	<code>StringVar</code> 등의 변수 객체	값과 연동할 <code>StringVar</code> 등의 변수를 지정
<code>width</code>	정수 (문자 단위)	위젯의 너비(문자 단위)를 지정
<code>height</code>	정수 (항목 수)	드롭다운 목록에 한 번에 표시할 항목 수를 지정
<code>justify</code>	<code>left</code> , <code>center</code> , <code>right</code>	입력 텍스트의 정렬 방식을 지정
<code>font</code>	폰트 지정 (예: <code>('맑은 고딕', 12)</code>)	표시 글꼴을 지정
<code>postcommand</code>	함수 또는 콜백 객체	드롭다운 목록이 열리기 직전에 호출할 함수를 지정

프로그레스바

- 작업의 진행 상황을 막대 형태로 시각화하는 위젯입니다.
- 진행률을 알 수 있을 때는 `determinate` 모드로 값을 채우고, 알 수 없을 때는 `indeterminate` 모드로 막대를 움직여 작업 중임을 표시합니다.
- 파일 다운로드, 설치, 데이터 처리 등 시간이 걸리는 작업의 상태를 사용자에게 알려줄 때 주로 사용합니다.

프로그레스바 사용하기

- 버튼을 클릭하면 Progressbar가 10%씩 증가합니다.

```

import tkinter as tk
from tkinter import ttk

root = tk.Tk()

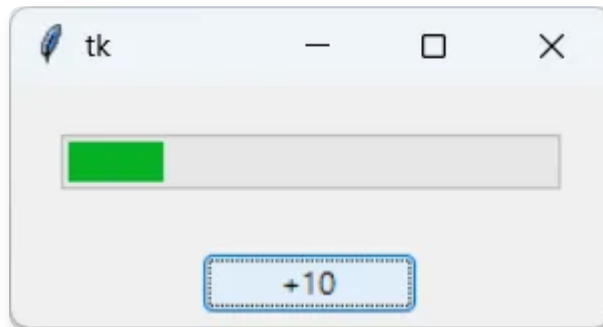
pb = ttk.Progressbar(root, length=200, mode="determinate", maximum=100)
pb.pack(padx=20, pady=20)
pb["value"] = 0

def step():
    pb["value"] = (pb["value"] + 10) % 110

ttk.Button(root, text="+10", command=step).pack(pady=5)

root.mainloop()

```



주요 메서드

메서드	설명
<code>start(interval=50)</code>	<code>indeterminate</code> 모드에서 애니메이션을 시작. <code>interval</code> 은 갱신 간격(ms)
<code>stop()</code>	애니메이션 중지
<code>step(amount=1.0)</code>	현재 값을 <code>amount</code> 만큼 증가 <code>maximum</code> 을 넘으면 0으로 순환

주요 옵션

옵션	허용 값	설명
<code>mode</code>	<code>determinate</code> , <code>indeterminate</code>	값 기반 또는 애니메이션 모드를 지정
<code>maximum</code>	실수 (기본값 <code>100</code>)	진행률의 최대값
<code>value</code>	실수 (0 ~ maximum)	현재 진행 값. <code>determinate</code> 모드에서 사용
<code>variable</code>	<code>IntVar</code> 또는 <code>DoubleVar</code> 변수 객체	진행 값을 담을 변수를 연결
<code>length</code>	정수 (픽셀)	진행바의 길이
<code>orient</code>	<code>horizontal</code> , <code>vertical</code>	방향을 지정함

노트북 (탭)

- 여러 페이지를 탭으로 묶어, 클릭으로 전환할 수 있게 해주는 위젯입니다.
- 각 탭에는 보통 Frame을 연결하고, 그 안에 원하는 위젯을 배치합니다.

노트북 으로 탭 기능 사용하기

- 두 개의 프레임을 '메인 탭', '설정 탭'으로 구성합니다.

```

import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.geometry("300x150")

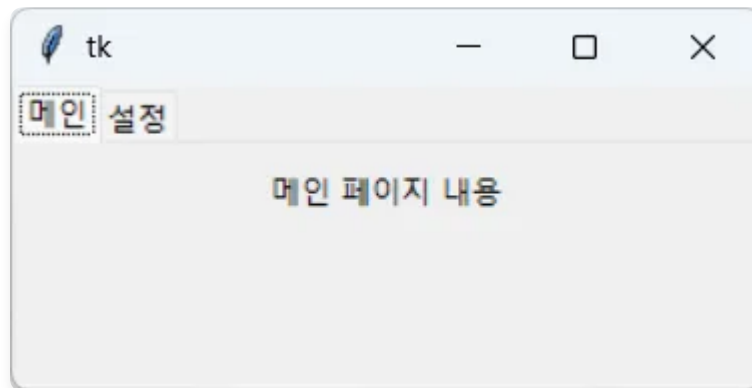
notebook = ttk.Notebook(root)
frame1 = ttk.Frame(notebook, width=300, height=150)
frame2 = ttk.Frame(notebook, width=300, height=150)

ttk.Label(frame1, text="메인 페이지 내용").pack(padx=10, pady=10)
ttk.Label(frame2, text="설정 페이지 내용").pack(padx=10, pady=10)

notebook.add(frame1, text="메인")
notebook.add(frame2, text="설정")
notebook.pack(expand=True, fill="both")

root.mainloop()

```



주요 메서드

메서드	설명
<code>add(child, [options])</code>	새 탭을 추가. <code>text</code> 로 탭 이름을 지정
<code>insert(index, child, [options])</code>	지정한 위치(<code>index</code>)에 탭을 삽입
<code>forget(tab_id)</code>	해당 탭을 제거
<code>select(tab_id)</code>	지정한 탭을 선택(전환). 인자가 없으면 현재 선택된 탭을 반환
<code>index(tab_id)</code>	탭의 인덱스를 반환. <code>"end"</code> 를 넘기면 탭 개수를 반환
<code>tab(tab_id, **options)</code>	특정 탭의 옵션을 조회하거나 변경

주요 옵션

옵션	허용 값	설명
<code>text</code>	문자열	탭에 표시할 이름 (<code>add</code> / <code>insert</code> 에서 사용)
<code>state</code>	<code>normal</code> , <code>disabled</code> , <code>hidden</code>	탭 상태를 지정
<code>image</code>	<code>PhotoImage</code> 등 이미지 객체	탭에 표시할 이미지
<code>compound</code>	<code>left</code> , <code>right</code> , <code>top</code> , <code>bottom</code> , <code>center</code>	텍스트와 이미지를 함께 배치할 때 위치를 지정
<code>padding</code>	정수 또는 (좌, 상, 우, 하) 형식	탭 내부 여백
<code>sticky</code>	<code>n</code> , <code>s</code> , <code>e</code> , <code>w</code> 의 조합	연결된 자식 위젯이 탭 영역에서 정렬·확장되는 방식을 지정

트리뷰

- 데이터를 행과 열로 표시하는 위젯으로, 표와 트리를 표현할 수 있습니다.
- 각 행은 항목(item)이며, 부모-자식 관계로 펼치고 접을 수 있는 트리 구조를 만들 수 있습니다.

트리뷰 사용하기

```

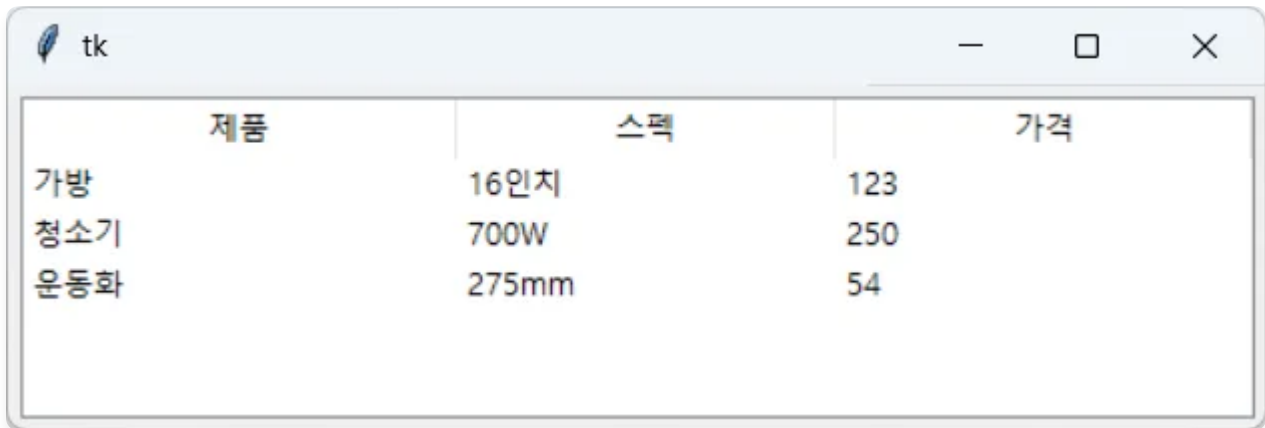
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
tree = ttk.Treeview(root, columns=("item", "spec", "price"), show="headings")
tree.heading("item", text="제품")
tree.heading("spec", text="스펙")
tree.heading("price", text="가격")

data = [("가방", "16인치", 123), ("청소기", "700W", 250), ("운동화", "275mm", 54)]
for item, spec, price in data:
    tree.insert("", "end", values=(item, spec, price))

tree.pack(padx=5, pady=5, expand=True, fill="both")
root.mainloop()

```



제품	스펙	가격
가방	16인치	123
청소기	700W	250
운동화	275mm	54

주요 메서드

메서드	설명
<code>insert(parent, index, **options)</code>	항목을 추가. <code>parent</code> 가 "" 이면 최상위 항목. 추가된 항목 ID 를 반환
<code>delete(*items)</code>	지정한 항목을 삭제
<code>heading(column, **options)</code>	열 머리글의 텍스트나 정렬 명령 등을 설정
<code>column(column, **options)</code>	열의 너비, 정렬 방식 등을 설정
<code>item(item, **options)</code>	특정 항목의 값이나 옵션을 조회·변경
<code>selection()</code>	현재 선택된 항목들의 ID를 반환
<code>set(item, column, value)</code>	특정 항목의 셀 값을 조회하거나 변경
<code>get_children(item="")</code>	해당 항목의 자식 항목 ID 목록을 반환

주요 옵션

옵션	허용 값	설명
<code>columns</code>	식별자 목록	표시할 열들의 식별자 목록
<code>show</code>	<code>tree</code> , <code>headings</code> , 또는 둘 다를 조합	표시 영역을 지정
<code>selectmode</code>	<code>browse</code> , <code>extended</code> (기본), <code>none</code>	선택 방식(단일/다중/비활성)
<code>height</code>	정수	한 번에 보이는 행의 개수
<code>text</code>	문자열	항목의 트리 열에 표시할 텍스트 (<code>insert</code> 에서 사용)
<code>values</code>	값 목록	항목의 각 열에 들어갈 값들 (<code>insert</code> 에서 사용)
<code>anchor</code>	<code>center</code> , <code>w</code> , <code>e</code> 등	열 내 텍스트 정렬 위치 (<code>column</code> 에서 사용)